

CKA: Networking

KodeKloud Complete Study Notes — Certified Kubernetes Administrator

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Networking Introduction

Overview of Kubernetes networking fundamentals including pod IP addressing, CNI plugins, services, Cluster DNS, load balancing, ingress, and troubleshooting.

Hello, and welcome to this section on Kubernetes networking. My name is Mumshad Mannambeth. This lesson covers the core networking concepts you need to design, operate, and troubleshoot Kubernetes clusters.

A solid understanding of a few foundational networking topics will help you get the most from this section. Before diving into cluster-specific details, make sure you are comfortable with:

- Configuring network interfaces and IP addresses
- Gateways and routing basics
- Name resolution and DNS fundamentals
- DNS configuration on Linux systems
- CoreDNS basics
- Network namespaces and how container runtimes (for example Docker) use them

These short prerequisite lectures are optional but recommended. If you already know these basics, skip what you don't need — but review network namespaces and Docker networking if you haven't, since they directly affect how pods and containers are isolated and connected.

The slide features a dark blue vertical bar on the left with the word 'Networking' in white. To its right is a list of seven prerequisites, each preceded by a teal diamond containing a white number. On the far right is a video of a man with a beard, wearing a dark blue sweater with orange stripes on the sleeves, gesturing with his hands.

Networking

- 01 Prerequisites – Switching and Routing
- 02 Prerequisites – CoreDNS
- 03 Prerequisites – Tools
- 04 Prerequisites – DNS
- 05 Prerequisites CNI
- 06 Prerequisites – Networking in Docker
- 07 Networking Configuration on Cluster Nodes

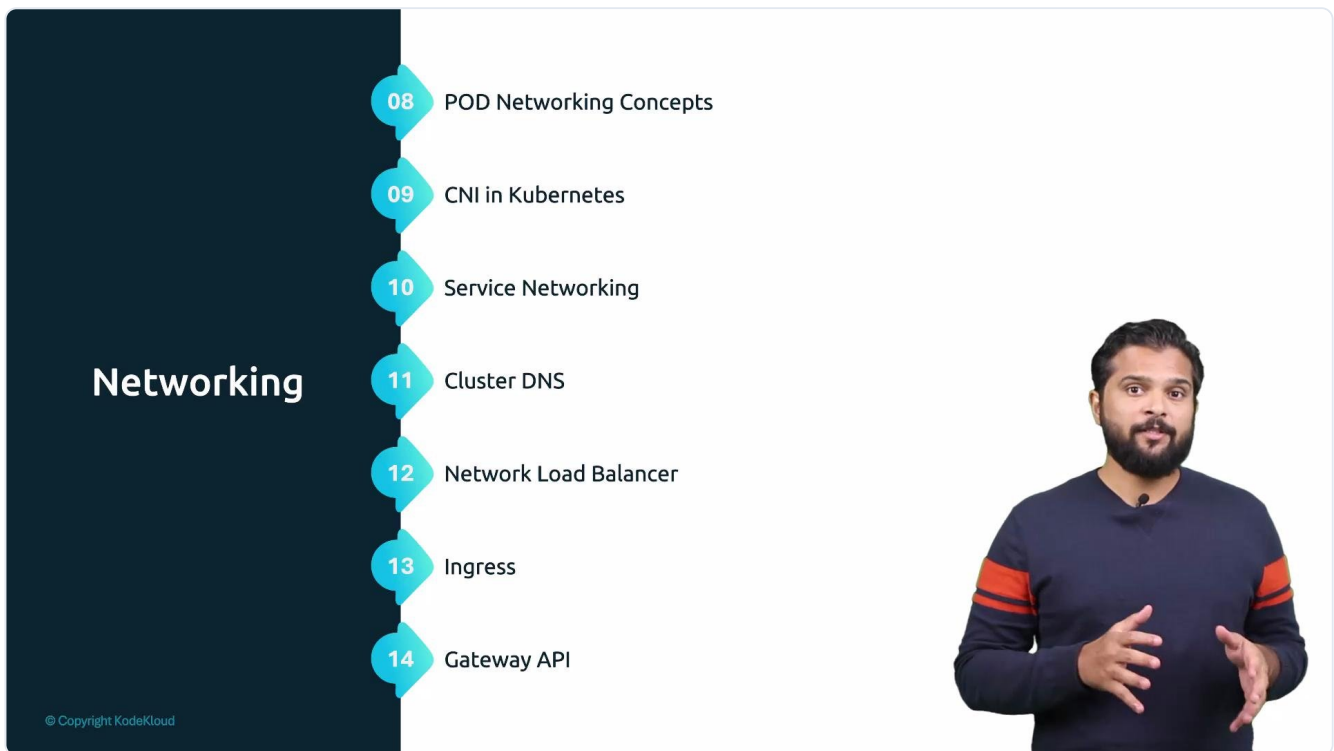
© Copyright KodeKloud

Why these prerequisites matter

- Network namespaces define per-process network stacks used by container runtimes; understanding them makes pod isolation and container networking much clearer.
- Gateways, routes, and interface configuration are essential when diagnosing connectivity problems between nodes, pods, and external services.
- DNS and CoreDNS are central to Kubernetes service discovery — misconfiguration here is a common source of application failures.

Learning sequence (what we'll cover) To provide a clear learning path, this lesson follows an ordered sequence that builds from cluster-level requirements to higher-level routing and ingress patterns:

1. Cluster networking needs — what Kubernetes expects from the network
2. Pod networking concepts — IP addressing, isolation, and reachability for pods
3. CNI in Kubernetes — how Container Network Interface plugins provide pod networking
4. Service networking — ClusterIP, NodePort, and stable endpoints for applications
5. Cluster DNS — how Kubernetes implements DNS (CoreDNS) for service discovery
6. Network load balancers — external access patterns and load balancing options
7. Ingress and Gateway API — HTTP routing and the newer Gateway API for advanced ingress



What you'll gain from this lesson

- A mental model of how Kubernetes connects pods, services, and external clients.
- Practical knowledge of CNI plugins and how they affect pod IP allocation and routing.
- Familiarity with Service types (ClusterIP, NodePort, LoadBalancer), when to use each, and common troubleshooting steps.
- Understanding of Cluster DNS (CoreDNS) patterns for service discovery and name resolution.
- An overview of ingress patterns, load balancers, and the Gateway API for modern HTTP routing.

Quick reference — concepts and resources

Concept	Purpose	Where to start
Pod networking	IP addressing and connectivity between containers	Pod CIDR, network namespaces
CNI	Plugin model for providing pod network connectivity	CNI plugins (Calico, Flannel, Cilium)
Service types	Stable access to pods: ClusterIP, NodePort, LoadBalancer	<code>kubectl get svc</code> and Service spec
Cluster DNS	Service discovery via CoreDNS	CoreDNS ConfigMap and <code>kube-dns</code>
Ingress / Gateway API	HTTP routing and advanced ingress features	Ingress controllers; Gateway API docs
Network namespaces	Process-level networking isolation	<code>ip netns</code> , container runtime networking

Further reading and references

- Kubernetes Networking Concepts: <https://kubernetes.io/docs/concepts/cluster-administration/networking/>
- CoreDNS: <https://coredns.io/>
- CNI (Container Network Interface): <https://github.com/containernetworking/cni>
- Kubernetes Services: <https://kubernetes.io/docs/concepts/services-networking/service/>
- Ingress and Gateway API: <https://kubernetes.io/docs/concepts/services-networking/ingress/> and <https://gateway-api.sigs.k8s.io/>

By the end of this lesson you should be able to reason about IP addressing and routing inside a cluster, choose appropriate CNI and Service types for common application topologies, and troubleshoot common DNS and ingress-related networking issues.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Prerequisite Switching Routing Gateways CNI in kubernetes

This article explores essential networking concepts for configuring Kubernetes environments, covering switching, routing, gateways, DNS, network namespaces, and Docker networking.

In this article, we explore essential networking concepts from a Linux perspective that are fundamental for configuring Kubernetes environments. We will cover topics such as switching, routing, gateways, DNS, network namespaces, and Docker networking. This guide targets both system administrators and application developers. If you are already confident with Linux networking, feel free to jump ahead to the Kubernetes-focused sections.

I Networking Pre-Requisites

- Switching and Routing
 - Switching
 - Routing
 - Default Gateway
- DNS
 - DNS Configurations on Linux
 - CoreDNS Introduction
- Network Namespaces
- Docker Networking

Basic Networking Concepts

Network Interfaces and Switching

Consider two systems—labeled A and B—that could be laptops, desktops, or virtual machines. To enable communication between them, each system must be connected to a switch with its respective network interface (either physical or virtual). To list available interfaces on a Linux host, execute:

```
``bash theme={null} ip link
```

A sample output might be:

```
``bash theme={null}
eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qdisc fq_codel
```

Assuming these systems belong to the network 192.168.1.0, you can assign IP addresses using the following command:

```
``bash theme={null} ip addr add 192.168.1.10/24 dev eth0
```

After configuration, test connectivity by pinging another host within the same network:

```
``bash theme={null}
ping 192.168.1.11
```

A successful ping might look like:

```
``bash theme={null} Reply from 192.168.1.11: bytes=32 time=4ms TTL=117
```

Routing Between Subnets

Now, consider a second network, such as 192.168.2.0, with hosts assigned IPs like 192.168.2.10 and 192.168.2.11.

A router interconnects two or more networks and holds an IP address in each network—e.g., 192.168.1.1 for the first network and 192.168.2.1 for the second.

<Frame>
![The image illustrates a network routing diagram with devices A, B, C, and D connected through two subnets. Subnet 1 (192.168.1.0/24) contains devices A and B, and Subnet 2 (192.168.2.0/24) contains devices C and D. A router with IP 192.168.1.1 in Subnet 1 and 192.168.2.1 in Subnet 2 connects the two subnets. Arrows show traffic flow from A to C and from B to D, passing through the router.]</Frame>

Each system must be configured with a gateway or specific route entries to ensure that packets reach the destination.

```
``bash theme={null}
route
```

Initially, communication will be limited to the same subnet. To route traffic destined for 192.168.2.0 via the router (with IP 192.168.1.1), add the following route:

```
``bash theme={null} ip route add 192.168.2.0/24 via 192.168.1.1
```

After adding the route, verifying the routing table should show an entry similar to:

```
```bash theme={null}
route
Kernel IP routing table
Destination Gateway Genmask Flags Metric Ref Use Iface
192.168.2.0 192.168.1.1 255.255.255.0 UG 0 0 0 eth0
```

If a return route is required (for instance, for a host in network 192.168.2.0 to reach a host in 192.168.1.0), add the appropriate route on that system using its corresponding gateway (e.g., 192.168.2.1).

## Configuring Default Routes for Internet Access

To enable internet access (such as reaching external hosts like 172.217.194.0), configure the router as the default gateway. This is done by adding a default route:

```
```bash theme={null} ip route add default via 192.168.2.1
```

Afterward, your routing table might resemble the following:

```
```bash theme={null}
route
Kernel IP routing table
Destination Gateway Genmask Flags Metric Ref Use Iface
192.168.1.0 192.168.2.1 255.255.255.0 UG 0 0 0 eth0
172.217.194.0 192.168.2.1 255.255.255.0 UG 0 0 0 eth0
default 192.168.2.1 0.0.0.0 UG 0 0 0 eth0
```

The "default" or "0.0.0.0" entry indicates that any destination not explicitly listed in the routing table will be directed through the specified gateway.

For scenarios involving multiple routers—such as one handling internet traffic and another managing internal networks—ensure each network has its specific routing entry along with a default route for all other traffic. For example, to route traffic to network 192.168.1.0 via an alternative gateway (192.168.2.2), use:

```
```bash theme={null} ip route add 192.168.1.0/24 via 192.168.2.2
```

The updated routing table should include:

```
```bash theme={null}
route
Kernel IP routing table
Destination Gateway Genmask Flags Metric Ref Use Iface
default 192.168.2.1 0.0.0.0 UG 0 0 0 eth0
192.168.1.0 192.168.2.2 255.255.255.0 UG 0 0 0 eth0
```

If you encounter internet connectivity issues, reviewing the routing table and default gateway configuration is a good troubleshooting practice.

## Configuring a Linux Host as a Router

Consider a scenario with three hosts (A, B, and C) where host B connects to two subnets (192.168.1.x and 192.168.2.x) using two interfaces. For example:

- **Host A:** 192.168.1.5
- **Host B:** 192.168.1.6 and 192.168.2.6
- **Host C:** 192.168.2.5

For host A to communicate with host C, host A must direct traffic aimed at network 192.168.2.0 to host B. On host A, execute:

```
``bash theme={null} ip route add 192.168.2.0/24 via 192.168.1.6
```

Similarly, host C needs a route for the 192.168.1.0 network via host B (using 192.168.2.6 as the gateway).

```
``bash theme={null}
ip route add 192.168.1.0/24 via 192.168.2.6
```

Once these routes are established, the "network unreachable" error should no longer occur when pinging between host A and host C.

### Enabling IP Forwarding on Linux

Even with the correct routing table, Linux does not forward packets between interfaces by default, as a security measure. This setting is controlled by the IP forwarding parameter in `/proc/sys/net/ipv4/ip_forward`.

To check the IP forwarding status, run:

```
``bash theme={null} cat /proc/sys/net/ipv4/ip_forward
```

A return value of `0` indicates that packet forwarding is disabled. To enable forwarding temporarily, run:

```
``bash theme={null}
echo 1 > /proc/sys/net/ipv4/ip_forward
```

Verifying again should now show:

```
``bash theme={null} cat /proc/sys/net/ipv4/ip_forward
```

with the output:

```
``bash theme={null}
1
```

To ensure this setting persists across reboots, modify `/etc/sysctl.conf` and add or update the following line:

```
text theme={null} net.ipv4.ip_forward = 1
```

Modifying `/etc/sysctl.conf` ensures that IP forwarding remains enabled even after a system restart.

## Summary of Key Commands

Below is a summary table of essential commands covered in this article:

Operation	Command Example
List network interfaces	<code>ip link</code>
View assigned IP addresses	<code>ip addr</code>
Assign an IP address	<code>ip addr add 192.168.1.10/24 dev eth0</code>
View the routing table	<code>route</code>
Add a specific route	<code>ip route add 192.168.1.0/24 via 192.168.2.1</code>
Set a default gateway	<code>ip route add default via 192.168.2.1</code>
Check IP forwarding status	<code>cat /proc/sys/net/ipv4/ip_forward</code>
Enable IP forwarding temporarily	<code>echo 1 &gt; /proc/sys/net/ipv4/ip_forward</code>

Remember, changes made with these commands are temporary and will be reset upon reboot unless they are saved in the appropriate configuration files.

That concludes this article. In the next installment, we will dive into DNS configurations and further enhance your understanding of Kubernetes networking.

For additional details and further reading, refer to resources like [Kubernetes Documentation](#) and [Docker Hub](#).

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

# Prerequisite DNS

This article introduces DNS for Linux beginners, covering basic concepts, configuration steps, and hands-on labs for understanding local name resolution and centralized DNS servers.

In this article, we introduce DNS for Linux beginners with an SEO-friendly guide that covers basic concepts, configuration steps, and hands-on labs to challenge your understanding of DNS. Learn how local name resolution works and how to transition from simple `/etc/hosts` setups to a full-blown centralized DNS server.

## Understanding Local Name Resolution

Imagine you have two computers on the same network—Computer A with IP address 192.168.1.10 and Computer B with IP address 192.168.1.11. You can easily ping Computer B from Computer A using its IP address:

```
```bash theme={null} ping 192.168.1.11 Reply from 192.168.1.11: bytes=32 time=4ms TTL=117 Reply from 192.168.1.11: bytes=32 time=4ms TTL=117
```

Suppose Computer B offers database services. Instead of remembering its IP address, you'll refer to it by

```
```bash theme={null}
ping db
ping: unknown host db
```

To make "db" recognizable, add an entry in the `/etc/hosts` file on Computer A. This informs the system that Computer B (192.168.1.11) is known as "db":

```
```bash theme={null} cat >> /etc/hosts 192.168.1.11 db
```

After this change, pings to "db" resolve correctly:

```
```bash theme={null}
ping db
PING db (192.168.1.11) 56(84) bytes of data.
64 bytes from db (192.168.1.11): icmp_seq=1 ttl=64 time=0.052 ms
64 bytes from db (192.168.1.11): icmp_seq=2 ttl=64 time=0.079 ms
```

Once you trust the mappings in `/etc/hosts`, the system does not verify whether the actual hostname (e.g., Computer B's real name) matches the alias you defined.

You can even create multiple aliases for a single IP address. For instance, you might convince Computer A that Computer B is also known as "[www.google.com](http://www.google.com)":

```
```bash theme={null} cat >> /etc/hosts 192.168.1.11 db 192.168.1.11 www.google.com
```

```
ping db PING db (192.168.1.11) 56(84) bytes of data. 64 bytes from db (192.168.1.11): icmp_seq=1 ttl=64 time=0.052 ms
64 bytes from db (192.168.1.11): icmp_seq=2 ttl=64 time=0.079 ms
```

```
ping www.google.com PING www.google.com (192.168.1.11) 56(84) bytes of data. 64 bytes from www.google.com (192.168.1.11): icmp_seq=1 ttl=64 time=0.052 ms
64 bytes from www.google.com (192.168.1.11): icmp_seq=2 ttl=64 time=0.079 ms
```

Every time you reference a host by name—whether by ping, SSH, or curl—the system consults the `/etc/hosts`

```
```bash theme={null}
cat >> /etc/hosts
192.168.1.11 db
192.168.1.11 www.google.com

ping db
ssh db
curl http://www.google.com
```

While managing local `/etc/hosts` files works for small networks, it becomes difficult to maintain as the number of systems grows and IP addresses change.

## Scaling with a Centralized DNS Server

To overcome the challenges of managing numerous local host mappings, organizations consolidate all mappings on a centralized DNS server. Suppose your centralized DNS server is at IP address 192.168.1.100. You configure each host to use this server by editing the `/etc/resolv.conf` file:

```
```bash theme={null} cat /etc/resolv.conf nameserver 192.168.1.100
```

Once configured, any hostname that is not found in `/etc/hosts` is resolved via the DNS server. If an IP

```
```bash theme={null}
cat /etc/nsswitch.conf
...
hosts: files dns
...
```

In this configuration, the system first searches the `/etc/hosts` file for a hostname. If a match is not found, it then queries the DNS server.

Now, if you try pinging a hostname not found in either `/etc/hosts` or the DNS server (e.g., [www.facebook.com](http://www.facebook.com)), the resolution fails:

```
```bash theme={null} cat >> /etc/hosts 192.168.1.115 test
```

```
cat /etc/nsswitch.conf ... hosts: files dns ...
```

```
ping www.facebook.com ping: www.facebook.com: Temporary failure in name resolution
```

To resolve external domains like Facebook, add a public DNS server (for example, Google's 8.8.8.8) or configure DNS forwarding.

Domain Names and Structure

Up until now, we have been resolving internal hostnames such as web, db, and nfs. But what is a domain name?

- The top-level domain (TLD) appears at the end (e.g., .com, .net, .edu, .org).\
- The domain name precedes the TLD (e.g., facebook in www.facebook.com).\
- Any segment before the domain name is considered a subdomain (e.g., www).

For instance, consider Google's domain:

- The root is implicit.\
- ".com" is the TLD.\
- "google" is the main domain.\
- "www" is a subdomain.

Subdomains allow organizations to separate services. Examples from Google include maps.google.com.

<Frame>

! [The image illustrates the structure of domain names, showing root, top-level domain (.com), main domain (google), and subdomain (www).]

</Frame>

When your organization attempts to access a domain like apps.google.com, the internal DNS server first tries to resolve the domain.

<Frame>

! [The image illustrates the DNS resolution process for "apps.google.com," showing the hierarchy from root to top-level domain (.com), main domain (google), and subdomain (apps).]

</Frame>

Similarly, organizations like mycompany.com can structure their domain by using subdomains for different services.

- * www.mycompany.com: External website
- * mail.mycompany.com: Email service
- * drive.mycompany.com: Storage solution
- * payroll.mycompany.com: Payroll systems
- * hr.mycompany.com: Human resources

Using Search Domains for Short Names

Within many organizations, it is often convenient to use short hostnames. To resolve a short name (for example, web), you can configure a search domain.

```
```bash theme={null}
```

```
cat >> /etc/resolv.conf
```

```
nameserver 192.168.1.100
```

```
search mycompany.com
```

```
ping web
```

```
PING web (192.168.1.10) 56(84) bytes of data.
```

```
64 bytes from web (192.168.1.10): icmp_seq=1 ttl=64 time=0.052 ms
```

```
64 bytes from web (192.168.1.10): icmp_seq=2 ttl=64 time=0.079 ms
```

Without the proper search domain, attempts to resolve "web" may fail:

```
```bash theme={null} ping web ping: web: Temporary failure in name resolution
```

```
ping web.mycompany.com PING web.mycompany.com (192.168.1.10) 56(84) bytes of data. 64 bytes from web.mycompany.com (192.168.1.10): ttl=64 time=0.052 ms
```

You can also specify multiple search domains. In the following example, the system will sequentially ap

```
```bash theme={null}
cat >> /etc/resolv.conf
nameserver 192.168.1.100
search mycompany.com prod.mycompany.com

ping web
PING web.mycompany.com (192.168.1.10) 56(84) bytes of data.
64 bytes from web.mycompany.com (192.168.1.10): icmp_seq=1 ttl=64 time=0.052 ms
64 bytes from web.mycompany.com (192.168.1.10): icmp_seq=2 ttl=64 time=0.079 ms

ping web.mycompany.com
PING web.mycompany.com (192.168.1.10) 56(84) bytes of data.
64 bytes from web.mycompany.com (192.168.1.10): ttl=64 time=0.052 ms

ping web.mvcompany.com
ping: web: Temporary failure in name resolution
```

## Overview of Common DNS Record Types

DNS records map hostnames to IP addresses and serve various other purposes. Here is an overview of some common DNS record types:

Record Type	Hostname	Address/Mapping
A	web-server	Maps hostname to an IPv4 address (e.g., 192.168.1.1)
AAAA	web-server	Maps hostname to an IPv6 address (e.g., 2001:0db8:85a3:0000:0000:8a2e:0370:7334)
CNAME	food.web-server	Aliases one hostname to another (e.g., aliasing to eat.web-server or hungry.web-server)

A records handle IPv4 addresses, AAAA records are for IPv6, and CNAME records allow hostname aliasing.

## Testing DNS Resolution Tools

While ping is the most common tool for verifying basic DNS resolution, utilities like `nslookup` and `dig` provide more detailed insights.

- The `nslookup` command does not consider `/etc/hosts` entries and only queries the configured DNS server.
- The `dig` command offers comprehensive details about DNS queries.

## Example: nslookup

```
```plaintext theme={null}
```

```
nslookup www.google.com Server: 8.8.8.8 Address: 8.8.8.8#53
```

Non-authoritative answer: Name: www.google.com Address: 172.217.0.132

```
### Example: dig

```bash theme={null}
dig www.google.com
; <<>> DiG 9.10.3-P4-Ubuntu <<>> www.google.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 28065
;; flags: qr rd ra; QUERY: 1, ANSWER: 6, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags;; udp: 512
;; QUESTION SECTION:
;www.google.com. IN A

;; ANSWER SECTION:
www.google.com. 245 IN A 64.233.177.103
www.google.com. 245 IN A 64.233.177.105
www.google.com. 245 IN A 64.233.177.147
www.google.com. 245 IN A 64.233.177.106
www.google.com. 245 IN A 64.233.177.104
www.google.com. 245 IN A 64.233.177.99

;; Query time: 5 msec
;; SERVER: 8.8.8.8#53(8.8.8.8)
;; WHEN: Sun Mar 24 04:34:33 UTC 2019
;; MSG SIZE rcvd: 139
```

## Next Steps

In upcoming practice exercises, you will configure and troubleshoot DNS on actual systems using tools like `dig` and `nslookup`. Later in the course, you will also learn how to set up an actual DNS server using CoreDNS as the DNS solution.

Thank you for reading this article. Enjoy exploring the intricacies of DNS and optimizing your network's name resolution process!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Prerequisite Network Namespaces

This article provides a comprehensive guide on network namespaces in Linux, focusing on network isolation and connectivity in containerized environments.

Welcome to this detailed guide on network namespaces in Linux. In this guide, we explain how network namespaces provide network isolation—a critical feature in containerized environments like Docker.

Imagine your host as a house and namespaces as the individual rooms. Each room isolates its occupant (the container), ensuring that processes and network interfaces remain private. While the container only sees the processes within its own namespace, the host maintains oversight over all namespaces and can bridge communication between them when required.

When a container is created, it is placed in its own network namespace. Inside this namespace, the container only sees its own processes. For example, inside a container, running:

```
```plaintext theme={null} ps aux USER PID %CPU %MEM VSZ RSS TTY STAT START TIME COMMAND root 1 0.0 0.0 4528 828 ? Ss 03:06 0:00 nginx
```

However, listing processes on the host as root shows all processes running on the system—including those

```
```plaintext theme={null}
ps aux
```

```
```plaintext theme={null} (On the container) USER PID %CPU %MEM VSZ RSS TTY STAT START TIME
COMMAND root 1 0.0 0.0 4528 828 ? Ss 03:06 0:00 nginx
```

```
(On the host) USER PID %CPU %MEM VSZ RSS TTY STAT START TIME COMMAND project 3720 0.1 0.1
95500 4916 ? R 06:06 0:00 sshd: project@pts/0 project 3725 0.0 0.1 95196 4132 ? S 06:06 0:00 sshd:
project@notty project 3727 0.2 0.1 21352 5340 pts/0 S 06:06 0:00 -bash root 3802 0.0 0.0 8924 3616 ? S 06:06
0:00 docker-containerd-shim -namespace m root 3816 1.0 0.0 4528 828 ? Ss 06:06 0:00 nginx
```

```
Notice that identical processes appear with different process IDs inside the container compared to the host
```

Network Isolation

On the networking front, the host maintains its own interfaces, ARP tables, and routing configurations—all

For example, running the following command on your host:

```
```bash theme={null}
ip link
```

displays the host's interfaces (such as the loopback and Ethernet interfaces). To examine interfaces within a specific network namespace (for example, the "red" namespace), use:

```
```bash theme={null} ip netns exec red ip link
```

Or with the shorthand using the `-n` option:

```
```bash theme={null}
ip -n red link
```

Inside the namespace, you typically see only a loopback interface, ensuring that host-specific interfaces (e.g., `eth0`) remain hidden. This isolation applies similarly to ARP and routing tables.

## Connecting Network Namespaces

By default, a network namespace has no connectivity because it lacks defined interfaces or routes. To connect namespaces, you create virtual Ethernet (veth) pairs that act like cables between two entities.

For example, to connect two namespaces—named "red" and "blue"—first create a veth pair:

```
```bash theme={null} ip link add veth-red type veth peer name veth-blue
```

Then, assign each end to its respective namespace:

```
```bash theme={null}
ip link set veth-red netns red
ip link set veth-blue netns blue
```

Next, assign IP addresses to each namespace's interface—for example, 192.168.15.1 for "red" and 192.168.15.2 for "blue"—and bring the interfaces up:

```
```bash theme={null} ip -n red addr add 192.168.15.1/24 dev veth-red ip -n blue addr add 192.168.15.2/24 dev veth-blue
```

```
ip -n red link set veth-red up ip -n blue link set veth-blue up
```

Test connectivity by pinging from the red namespace to the blue namespace:

```
```bash theme={null}
ip netns exec red ping 192.168.15.2
```

Verify neighbor discovery using ARP from within the red namespace:

```
```bash theme={null} ip netns exec red arp
```

Expected output:

```
```plaintext theme={null}
Address HWtype HWaddress Flags Mask Iface
192.168.15.2 ether ba:b0:6d:68:09:e9 C veth-red
```

Similarly, checking the ARP table in the blue namespace should display an entry for the red namespace. Note that these internal veth interfaces do not appear in the host's ARP table.

## Creating a Virtual Switch for Multiple Namespaces

When working with more than two namespaces, linking every pair using veth pairs is impractical. Instead, establish a virtual network switch (or bridge) on the host to interconnect all namespaces. Linux offers tools such as the native Linux bridge or Open vSwitch.

In this example, a Linux bridge is created:

1. Create a new bridge interface (named `v-net-0`):

```
bash theme={null} ip link add v-net-0 type bridge
```

1. Bring the bridge interface up:

```
bash theme={null} ip link set v-net-0 up
```

Namespaces can now be connected to this bridge. Remove the earlier direct veth pair as it is no longer necessary:

```
```bash theme={null} ip -n red link del veth-red
```

Next, create new veth pairs to link each namespace to the bridge. For example, create a pair with one end

```
```bash theme={null}
ip link add veth-red type veth peer name veth-red-br
ip link add veth-blue type veth peer name veth-blue-br
```

Attach the veth interfaces to their corresponding namespaces and the bridge:

```
```bash theme={null} ip link set veth-red netns red ip link set veth-red-br master v-net-0
```

```
ip link set veth-blue netns blue ip link set veth-blue-br master v-net-0
```

Assign IP addresses and activate the interfaces inside each namespace:

```
```bash theme={null}
ip -n red addr add 192.168.15.1/24 dev veth-red
ip -n blue addr add 192.168.15.2/24 dev veth-blue

ip -n red link set veth-red up
ip -n blue link set veth-blue up
```

To allow the host to communicate with the namespaces, assign an IP from the same subnet to the bridge interface:

```
```bash theme={null} ip addr add 192.168.15.5/24 dev v-net-0
```

You can now verify connectivity from the host, for example:

```
```bash theme={null}
ping 192.168.15.1
```

All traffic within this network remains private to the host, ensuring that namespaces are not directly accessible from external networks.

## Enabling External Connectivity

The internal network created by namespaces and a bridge is isolated from external networks. Suppose your host's external interface utilizes a LAN subnet (e.g., 192.168.1.0/24) and you wish for namespaces to communicate with external systems. For instance, pinging an external host (192.168.1.3) from the blue namespace would initially result in a "Network is unreachable" error:

```
```bash theme={null} ip netns exec blue ping 192.168.1.3
```

Checking the routing table in the blue namespace using:

```
```bash theme={null}
ip netns exec blue route
```

might display:

```
```plaintext theme={null} Destination Gateway Genmask Flags Metric Ref Use Iface 192.168.15.0 0.0.0.0
255.255.255.0 U 0 0 0 veth-blue
```

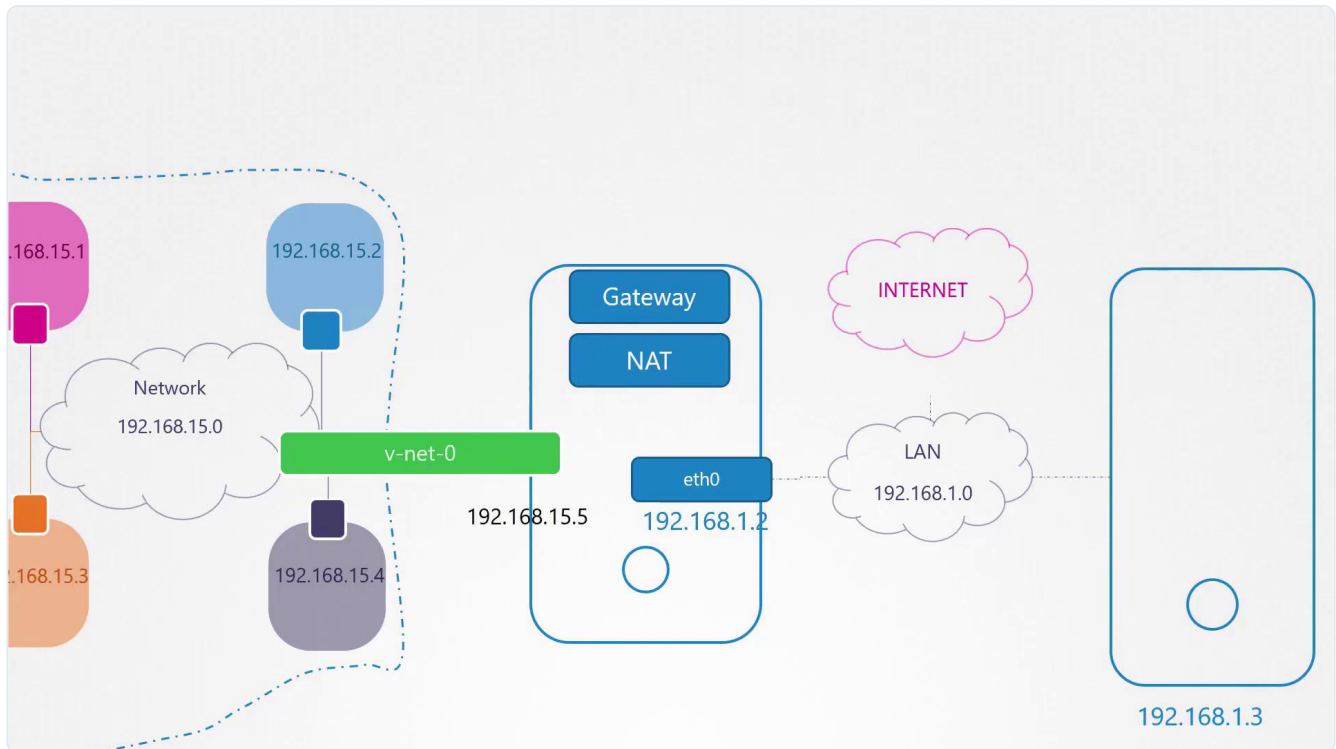
To enable external connectivity, add a route in the blue namespace that directs traffic destined for 192.168.1.0/24 via 192.168.15.5

```
```bash theme={null}
ip netns exec blue ip route add 192.168.1.0/24 via 192.168.15.5
```

After this route is added, the blue namespace can reach external networks. However, external networks will not recognize the private IP addresses. To address this, configure NAT on the host using iptables. This setup

masquerades packets from the 192.168.15.0/24 network with the host's external IP, making the traffic routable on the LAN.

To provide namespaces with Internet access (such as pinging 8.8.8.8), add a default route in the namespace pointing to the host; the host must then perform NAT to forward the traffic correctly.



## Enabling Inbound Access

Thus far, namespaces are isolated on an internal network and are not directly accessible from external hosts. For instance, if the blue namespace runs a web application on port 80, external users cannot access it simply by targeting its private IP.

There are two common solutions:

1. Add a static route on the external router or host so that traffic for the 192.168.15.0/24 network is directed through the host's LAN IP (e.g., 192.168.1.2).
2. Use port forwarding via iptables on the host to redirect traffic arriving at a specific port (e.g., port 80) to the corresponding port in the namespace.

The port forwarding method is often preferred, as it eliminates the need to reconfigure external routing.

This concludes our guide on network namespaces. By following these steps, you can effectively isolate, connect, and manage network namespaces for containerized environments. Thank you for reading!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Prerequisite Docker Networking

This article provides an overview of Docker networking options, including modes like none, host, and bridge, and explains container connectivity and port mapping.

Welcome to this comprehensive lesson on Docker networking. In this guide, we explore the various networking options available in Docker and how they relate to network namespaces. We'll begin by focusing on a single Docker host—a server with Docker installed. Assume the host has an Ethernet interface (eth0) with the IP address 192.168.1.10 on the local network.

## Docker Networking Modes

When you run a container, Docker provides several networking modes to tailor connectivity to your application's needs:

### None Network

With the "none" network mode, the container is not attached to any network. As a result, it cannot communicate with external systems, nor can external systems reach the container. For example:

```
```bash theme={null} docker run --network none nginx
```

<Callout icon="lightbulb" color="#1CB2FE">

Multiple containers started in the none network mode remain completely isolated from each other and the host.

</Callout>

Host Network

In the host network mode, the container shares its host's network stack. This means there is no network isolation.

```
```bash theme={null}
docker run --network host nginx
```

### Bridge Network

The default Docker networking mode is the bridge network. When Docker is installed, it automatically creates an internal private network called "bridge" (visible as "docker0" on the host) with a default subnet (usually 172.17.0.0/16). Each container connected to this network receives a unique IP address from this subnet. For example, running two containers:

```
```bash theme={null} docker run nginx docker run nginx
```

These containers communicate with each other over the internal bridge network. To inspect the list of available

```
```bash theme={null}
docker network ls
```

An example output might look like:

```
```text theme={null} NETWORK ID NAME DRIVER SCOPE 2b6008726112 bridge bridge local 0beb4870b093
host host local 99035e02694f none null local
```

Internally, Docker creates the "docker0" interface on the host, serving as the bridge between host and container

```
```bash theme={null}
ip link show docker0
```

By default, the "docker0" interface is assigned the IP address 172.17.0.1. When a container is launched, Docker creates a new network namespace for it (similar to those discussed in earlier lessons). To list the available network namespaces (a minor hack may be required to display Docker-created namespaces):

```
```bash theme={null} ip netns list
```

Typically, the namespace will have a name starting with an identifier like `b3165...`. To view the network

```
```bash theme={null}
docker inspect <container_id>
```

A snippet of the output under "NetworkSettings" might appear as follows:

```
```json      theme={null}      "NetworkSettings":      {      "Bridge":      "",      "SandboxID":
"b3165c10a92b50edc4c8aa5f37273e180907ded31", "SandboxKey": "/var/run/docker/netns/b3165c10a92b" }
```

Container Attachment to the Bridge Network

Docker attaches each container to the bridge network by creating a pair of virtual interfaces—essentially

Inspect the interfaces on the Docker host using:

```
```bash theme={null}
ip link show
```

You might see an output similar to:

```
```text theme={null} 4: docker0: mtu 1500 qdisc noqueue state UP mode DEFAULT group default link/ether
02:42:9b:5f:d6:21 brd ff:ff:ff:ff:ff:ff 8: vethbb1c343@if7: mtu 1500 qdisc noqueue master docker0 state UP mode
DEFAULT group default link/ether 9e:71:37:83:9f:50 brd ff:ff:ff:ff:ff:ff link-netnsid 1
```

```
To inspect the network namespace of a container (for example, with namespace ID `b3165c10a92b`), run:

```bash theme={null}
ip -n b3165c10a92b link show
```

The output might be similar to:

```
```text theme={null} 7: eth0@if8: mtu 1500 qdisc noqueue state UP mode DEFAULT group default link/ether
02:42:ac:11:00:03 brd ff:ff:ff:ff:ff:ff link-netnsid 0
```

To view the IP address assigned to the container's network interface:

```
```bash theme={null}
ip -n b3165c10a92b addr show eth0
```

This may display:

```
```text theme={null} 7: eth0@if8: mtu 1500 qdisc noqueue state UP group default link/ether 02:42:ac:11:00:03 brd
ff:ff:ff:ff:ff:ff link-netnsid 0 inet 172.17.0.3/16 brd 172.17.255.255 scope global eth0 valid_lft forever preferred_lft
forever
```

Each time a new container is created, Docker follows these sequential steps:

1. Creates a new network namespace.
2. Establishes a pair of virtual interfaces.
3. Attaches one end to the container's namespace and the other to the "docker0" bridge.
4. Assigns an IP address to the container's interface.

The virtual interface pairs are numbered consistently, with odd and even numbers forming a pair (e.g., 7 and 8).

Port Mapping

Consider a scenario where a container runs an nginx web application that listens on port 80. By default, a

For example, to map port 8080 on the host to port 80 within the container:

```
```bash theme={null}
docker run -p 8080:80 nginx
```

After setting up port mapping, you can test connectivity as follows:

- Accessing the container directly via its IP on port 80 might result in a failure:

```
bash theme={null} curl http://172.17.0.3:80 # Output: curl: (7) Failed to connect... No route to
host
```

- However, accessing the application using the host's IP and port 8080 should succeed:

```
bash theme={null} curl http://192.168.1.10:8080 # Output: Welcome to nginx!
```

All incoming traffic to port 8080 on the Docker host is forwarded to port 80 in the container.

## How Port Forwarding Works

Docker employs IP tables to implement port forwarding. This mechanism adds a Network Address Translation (NAT) rule to translate traffic arriving on a specific host port to the corresponding container port. For example, Docker might add a rule similar to the following:

```
```bash theme={null} iptables \ -t nat \ -A PREROUTING \ -j DNAT \ --dport 8080 \ --to-destination 80
```

This rule directs traffic arriving at port 8080 on the host to port 80 on the container. You can [review](#)

```
```bash theme={null}  
iptables -nvL -t nat
```

An excerpt from the output might resemble:

```
```text theme={null} Chain DOCKER (2 references) target prot opt source destination RETURN all -- anywhere  
anywhere DNAT tcp -- anywhere anywhere tcp dpt:8080 to:172.17.0.2:80
```

After the mapping is configured, testing access to the container using:

```
```bash theme={null}  
curl http://172.17.0.3:80
```

should return a response like:

```
text theme={null} Welcome to nginx!
```

## Conclusion

In summary, Docker networking offers multiple modes—none, host, and bridge—to manage connectivity for containers. The default bridge network uses a virtual switch (docker0) to connect containers via dedicated network namespaces, while port mapping enables external access by forwarding traffic from a designated host port to the container's port.

For a deeper dive into container networking, consider exploring the [Kubernetes Networking Basics](#) and other resources in the [Docker Documentation](#).

Happy learning!

## Documentation Index

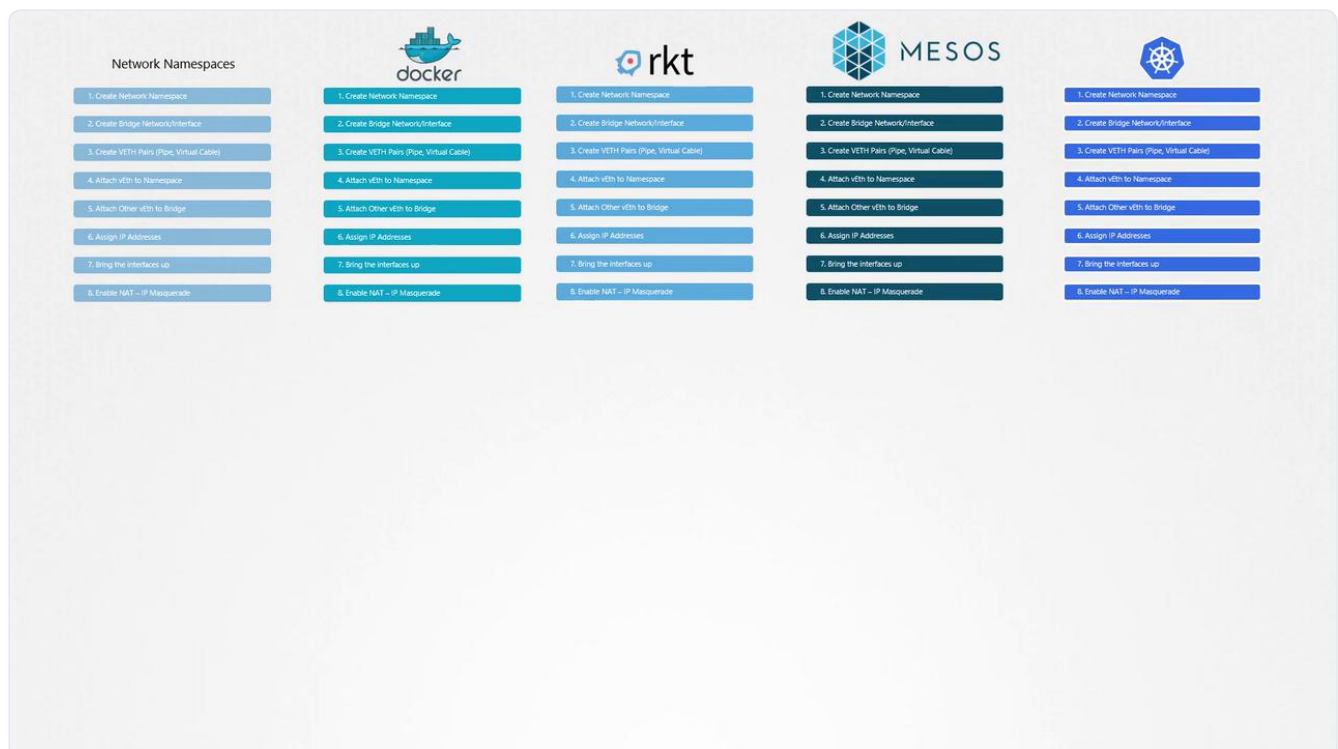
Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

## Prerequisite CNI

This article provides a comprehensive guide on the Container Networking Interface (CNI) and its role in simplifying container networking configuration and management.

Welcome to this comprehensive guide on the Container Networking Interface (CNI) and its vital role in container networking. In this lesson, we explore how network namespaces and standardized networking plugins simplify the configuration and management of container networks.

Network namespaces create isolated network environments on a single host. These namespaces are interconnected by a bridge network that establishes virtual interfaces (or virtual cables) for communication between namespaces. This involves assigning IP addresses, activating interfaces, and enabling NAT or IP masquerading for external connectivity. Although Docker configures its bridge networking using similar methods, it employs its own naming conventions. Other container platforms like Rocket, Mesos Containerizer, and Kubernetes address these networking challenges in a comparable way.



To standardize this process and avoid duplicating efforts across multiple platforms, a dedicated program known as "bridge" was developed. This program automates the tasks required to connect a container to a bridge network. For instance, you can run the program with the container ID and network namespace as shown below:

```
```bash theme={null} bridge add 2e34dcf34 /var/run/netns/2e34dcf34
```

The "bridge" program handles low-level networking configuration, freeing container runtime environments

<Callout icon="lightbulb" color="#1CB2FE">

By offloading network configuration tasks to a standardized bridge program, container runtimes can focus
</Callout>

This brings us to an important question: if you want to develop a similar program for a different network

CNI defines a standard for creating and integrating network plugins with container runtime environmen

- * Creating a network namespace for each container.
- * Identifying the networks to which the container should connect.
- * Configuring the network when a container is created (using the "add" command) and cleaning up when it is
- * Setting up necessary network details via a JSON configuration file.

On the plugin side, CNI requires support for three command-line arguments: "add", "del", and "check". The

<Frame>

! [The image outlines the Container Network Interface (CNI) requirements and processes, including network
</Frame>

When both container runtimes and network plugins adhere to CNI standards, seamless interoperability is a

<Callout icon="triangle-alert" color="#FF6B6B">

Docker uses its own networking standard known as the Container Network Model (CNM), which differs from C
</Callout>

Consider the following example that demonstrates how Kubernetes handles networking with Docker:

```
```bash theme={null}
docker run --network=none nginx
bridge add 2e34dcf34 /var/run/netns/2e34dcf34
```

In this workflow, Kubernetes first creates a Docker container without any network configuration and then calls the CNI plugin to establish the network. This process highlights how Kubernetes efficiently leverages CNI standards to manage container networks.

For further information, explore these resources:

- [Kubernetes Basics](#)
- [Kubernetes Documentation](#)
- [Docker Hub](#)
- [Terraform Registry](#)

Dive deeper into container networking and harness the power of CNI standards in your deployments. Happy networking!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

# Cluster Networking

This article explores networking configurations for master and worker nodes in a Kubernetes cluster, including required ports and verification commands.

In this article, we explore the networking configurations necessary for both the master and worker nodes within a Kubernetes cluster. Each node must be equipped with at least one network interface configured with an IP address. Additionally, every host should have a unique hostname and MAC address—this is especially important when creating virtual machines (VMs) by cloning existing ones.

## Required Ports for Kubernetes Components

Effective communication among the control plane components and worker nodes relies on specific port configurations. The following table summarizes the key ports that must be open:

Port Range	Component	Description
6443	Kubernetes API Server (master)	Used by worker nodes, the kube-controller-manager, external users, and other control plane components to access the API Server.
10250	Kubelet (master and worker)	Monitors cluster activities and manages nodes.
10259	Kube-scheduler (master)	Required for scheduling operations.
10257	Kube-controller-manager (master)	Needed for managing cluster state and various controllers.
30000–32767	Worker Nodes Services	Exposes services for external access on worker nodes.
2379 & 2380	etcd Server (master)	Port 2379 is used for client communication, and port 2380 is used for communication between etcd servers in multi-master deployments.

For a comprehensive list of required ports and additional configuration details, refer to the [Kubernetes Documentation](#). These details are crucial when configuring firewalls, IP tables, or network security groups across cloud platforms like GCP, Azure, or AWS.

## Verifying Network Configuration

---

To ensure that your cluster's network environment is set up correctly, it is useful to run several common commands. These commands help you inspect interfaces, IP addresses, hostnames, routing tables, and active services:

```
bash theme={null} ip link ip addr ip addr add 192.168.1.10/24 dev eth0 ip route ip route add 192.168.1.0/24 via 192.168.2.1 cat /proc/sys/net/ipv4/ip_forward arp netstat -plnt
```

These commands are invaluable for gathering information about your network interfaces, IP configurations, and port usage. As you continue to explore your Kubernetes cluster setup, these tools will assist you in troubleshooting and ensuring network connectivity.

In upcoming exercises, we will delve into more complex networking scenarios to further enhance your understanding of Kubernetes networking. For now, take your time to review the current configuration and familiarize yourself with the fundamental network requirements of your cluster.

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Pod Networking

This article explains pod networking in Kubernetes, covering implementation, inter-pod communication, and the role of CNI in managing network interfaces.

Welcome to this deep dive into pod networking in Kubernetes. In this article, we explain how pod networking is implemented within a Kubernetes cluster, detailing the steps that make inter-pod communication seamless across multiple nodes.

Kubernetes clusters consist of multiple master and worker nodes with pre-configured networking that allows full node-to-node communication. We assume that all Kubernetes control plane components (e.g., kube-apiserver, etcd, and kubelet) are already set up properly. With this foundation in place, the next step is deploying your applications with a focus on robust pod-level networking.

\* Each pod gets a unique IP address. \* Every pod on a node can reach every other pod on that node using its IP address. \* Pods across different nodes can communicate using a consistent addressing scheme.

These requirements ensure that Kubernetes can support both local (node-level) and cross-node communications without relying on NAT rules.

## Understanding the Basics

Before deploying applications, it is crucial to address several questions:

- How are pods assigned IP addresses?
- How do these pods communicate both within a node and across nodes?
- How can services running in these pods be accessed from inside or outside the cluster?

Kubernetes leaves the implementation details for pod networking to the user, as long as the chosen solution meets the basic connectivity requirements. Many networking solutions are available; here, we demonstrate the core concepts using Linux network namespaces, bridge networks, and IP address management inspired by CNI concepts.

## Setting Up a Simple Bridge Network

Below is an example that sets up a simple bridge network and connects network namespaces:

```
``bash theme={null} ip link add v-net-0 type bridge ip link set dev v-net-0 up ip addr add 192.168.15.5/24 dev v-net-0 ip link add veth-red type veth peer name veth-red-br ip link set veth-red netns red ip -n red addr add
```

```
192.168.15.1 dev veth-red ip -n red link set veth-red up ip link set veth-red-br master v-net-0 ip netns exec blue ip
route add 192.168.1.0/24 via 192.168.15.5 iptables -t nat -A POSTROUTING -s 192.168.15.0/24 -j
MASQUERADE
```

Imagine a cluster with three nodes where every node runs a combination of management and workload pods. A

### Cluster Network Planning

Consider the following plan for a three-node cluster:

1. **External Network Configuration**: Each node has an IP in the 192.168.1.0 series (e.g., 192.168.1.1).
2. **Container Network Namespaces**: Kubernetes creates unique network namespaces when containers start.
3. **Bridge Networks on Each Node**: Create a bridge network on every node. Assign a distinct subnet to each node's bridge.
  - \* Node one: 10.244.1.0/24
  - \* Node two: 10.244.2.0/24
  - \* Node three: 10.244.3.0/24

When a container is created, a virtual cable (veth pair) connects its network namespace to the node's bridge.

Below is a sample snippet from a script that connects a container to the network:

```
```bash theme={null}
# Create veth pair
ip link add <veth-in-host> type veth peer name <veth-in-namespace>
# Attach one end to the container's namespace
ip link set <veth-in-namespace> netns <namespace>
# Attach the other end to the bridge
ip link set <veth-in-host> master <bridge>
# Assign IP address to the container's interface
ip -n <namespace> addr add <IP-address>/24 dev <veth-in-namespace>
# Add default route in the container's namespace
ip -n <namespace> route add default via <bridge-IP>
# Bring up the container's interface
ip -n <namespace> link set <veth-in-namespace> up
```

Executing this script on each node ensures that all containers receive an IP address and are connected to their respective internal networks.

Cross-Node Communication

One of the challenges is enabling communication between pods on different nodes. For example, if a pod with IP 10.244.1.2 on node 1 attempts to ping a pod with IP 10.244.2.2 on node 2, the ping may initially fail due to unknown routes between subnets:

```
```bash theme={null} bluepod$ ping 10.244.2.2 Connect: Network is unreachable
```

To resolve this, add a route on node 1 that directs traffic for 10.244.2.2 via node 2's external IP (e.g., 192.168.1.12).

```
```bash theme={null}
node1$ ip route add 10.244.2.2 via 192.168.1.12
```

After configuring the routing, the ping command should succeed:

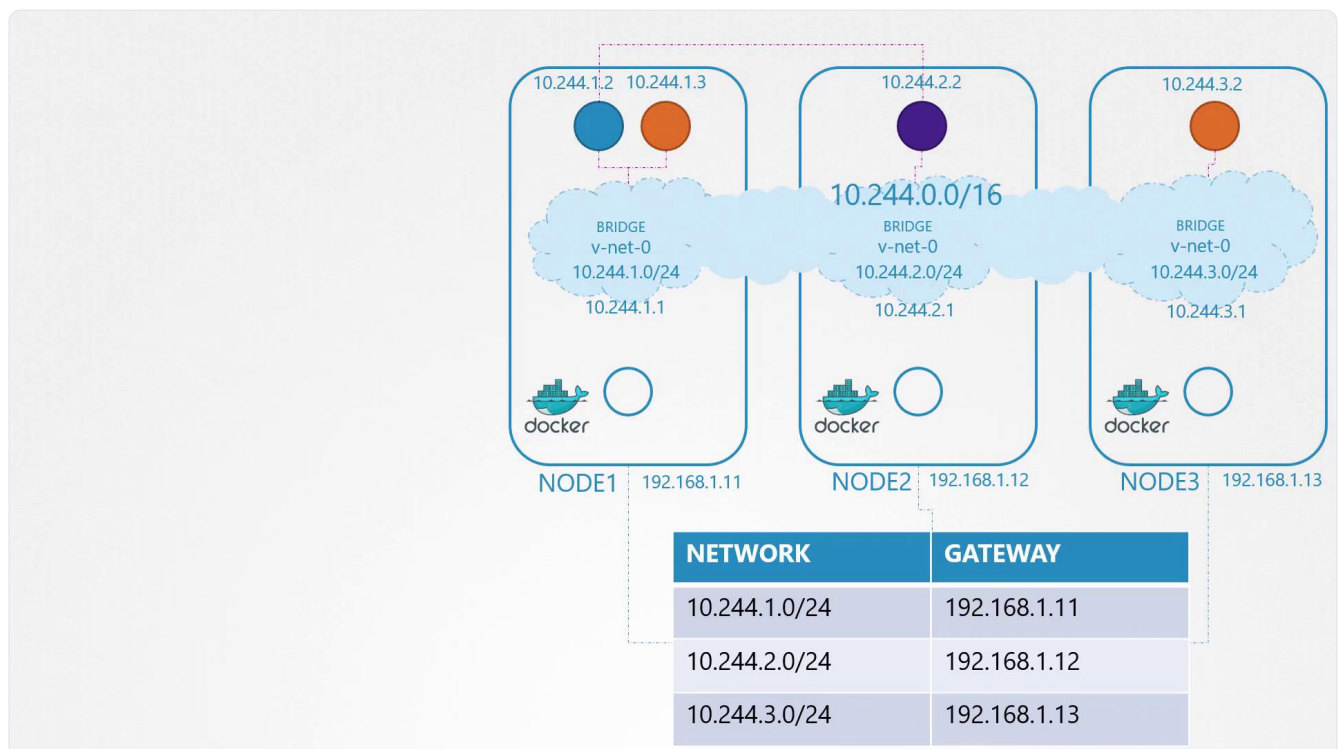
```
```bash theme={null} bluepod$ ping 10.244.2.2 64 bytes from 10.244.2.2: icmp_seq=1 ttl=63 time=0.587 ms 64
bytes from 10.244.2.2: icmp_seq=2 ttl=63 time=0.466 ms
```

Similarly, add these routes on all nodes to cover all pod subnets:

```
```bash theme={null}
node1$ ip route add 10.244.2.2 via 192.168.1.12
node1$ ip route add 10.244.3.2 via 192.168.1.13
node2$ ip route add 10.244.1.2 via 192.168.1.11
node2$ ip route add 10.244.3.2 via 192.168.1.13
node3$ ip route add 10.244.1.2 via 192.168.1.11
node3$ ip route add 10.244.2.2 via 192.168.1.12
```

Manually configuring routes on every host is impractical for large-scale deployments. A more scalable solution involves configuring a centralized router to manage all subnet routes and setting each node's default gateway to this router.

Below is an image that illustrates a Docker network setup with three nodes, each with distinct IP addresses and subnet configurations, connected via a virtual network bridge:



Integrating Container Network Interface (CNI)

In our lab setup, we executed scripts manually to configure pod networking. However, in a production Kubernetes environment where thousands of pods may be created per minute, this manual approach is not feasible.

This is where the Container Network Interface (CNI) becomes essential. CNI specifies how Kubernetes should invoke a networking script each time a pod is created. To conform with CNI standards, the networking script must have:

- An "add" section to connect the container to the network.

- A "delete" section to disconnect the container, remove interfaces, and free up the IP address.

When the container runtime launches a container, it uses the CNI configuration (provided as a command-line argument) to execute the relevant script with the command "add" and pass the container's name and namespace identifier.

Below is an example snippet that illustrates the CNI execution process:

```
bash theme={null} ip -n <namespace> link set <interface> up ip link del <interface> ./net-script.sh add <container> <namespace>
```

Conclusion

In this article, we covered the essential concepts behind pod networking in Kubernetes—from manual network namespace and bridge configuration to the role of CNI in automating network interface management. The techniques discussed here lay a solid foundation for understanding and troubleshooting pod networking in a Kubernetes cluster.

Stay tuned for upcoming articles where we integrate detailed CNI configurations into Kubernetes workflows and provide practical tests to reinforce your learning.

For more insight into Kubernetes networking, visit the [Kubernetes Documentation](#).

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

CNI in kubernetes

This lesson explores how Kubernetes uses the Container Network Interface to manage container networking and configure network plugins.

In this lesson, we explore how Kubernetes leverages the Container Network Interface (CNI) to manage container networking. You will gain an understanding of how network plugins are configured and used in a Kubernetes environment.

Earlier, we reviewed:

- The basics of networking and namespaces
- Docker networking fundamentals
- The evolution and rationale behind CNI
- A list of supported CNI plugins

Pre-Requisites

- ✓ Network Namespaces in Linux
- ✓ Networking in Docker
- ✓ Why and what is Container Network Interface (CNI)?
- ✓ CNI Plugins

Now, we focus on how Kubernetes configures the use of these network plugins.

As discussed in previous lessons, the CNI specifies the responsibilities of the container runtime. In Kubernetes, container runtimes such as Containerd or CRI-O create the container network namespaces and attach them to the correct network by invoking the appropriate network plugin. Although Docker was initially the primary container runtime, it has largely been replaced by Containerd as an abstraction layer.



CONTAINER NETWORK INTERFACE

- ✓ Container Runtime must create network namespace
- ✓ Identify network the container must attach to



Configuring CNI Plugins in Kubernetes

When a container is created, the container runtime invokes the necessary CNI plugin to attach the container to the network. Two common runtimes that demonstrate how this process works are Containerd and CRI-O.

Container runtimes look for CNI plugin executables in the `/opt/cni/bin` directory, while network configuration files are read from the `/etc/cni/net.d` directory.

Directory Structure for CNI Plugins and Configuration

The network plugins reside in `/opt/cni/bin`, and the configuration files that dictate which plugin to use are stored in `/etc/cni/net.d`. Typically, the container runtime selects the configuration file that appears first in alphabetical order.

For example, you might see the following directories:

```
```bash theme={null} ls /opt/cni/bin
```

```
```bash theme={null}
bridge dhcp flannel host-local ipvlan loopback macvlan portmap ptp sample tuning vlan weave-1p
```

```
```bash theme={null} ls /etc/cni/net.d
```

```
```bash theme={null}
10-bridge.conflist
```

In this case, the container runtime chooses the "bridge" configuration file.

Configuring CNI



Understanding a CNI Bridge Configuration File

A typical CNI bridge configuration file, adhering to the CNI standard, might look like this:

```
```bash theme={null} cat /etc/cni/net.d/10-bridge.conf
```

```
```json theme={null}
{
  "cniVersion": "0.2.0",
  "name": "mynet",
  "type": "bridge",
  "bridge": "cni0",
  "isGateway": true,
  "ipMasq": true,
  "ipam": {
    "type": "host-local",
    "subnet": "10.22.0.0/16",
    "routes": [
      { "dst": "0.0.0.0/0" }
    ]
  }
}
```

In this configuration:

- The `"name"` field (e.g., `"mynet"`) represents the network name.

- The `"type"` field set to `"bridge"` indicates the use of a bridge plugin.
- The `"bridge"` field (e.g., `"cni0"`) specifies the network bridge's name.
- The `"isGateway"` flag designates whether the bridge interface should have an IP address to function as a gateway.
- The `"ipMasq"` option enables network address translation (NAT) through IP masquerading.
- The `"ipam"` (IP Address Management) section uses `"host-local"` to allocate IP addresses from the specified subnet (`"10.22.0.0/16"`) and defines a default route.

Understanding these configuration fields is crucial for troubleshooting and optimizing Kubernetes networking. The settings in this bridge configuration align with fundamental networking concepts such as bridging, routing, and NAT masquerading.

This concludes our lesson on configuring CNI plugins with Kubernetes. We encourage you to apply these concepts through practical exercises to strengthen your Kubernetes networking skills.

For more information on related topics, consider reviewing:

- [Kubernetes Basics](#)
- [Kubernetes Documentation](#)
- [Docker Hub](#)

Happy networking!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Service Networking

This article explains service networking in Kubernetes, covering pod and service communication, service types, and how traffic is managed within a cluster.

Welcome to this comprehensive lesson on service networking. In this discussion, we explore how services function within a Kubernetes cluster and how they differ from pods. We'll also cover the inner workings of pod networking, including bridging, namespace assignments, interface attachments, and IP address management. This guide is designed to clarify how traffic flows through services to access applications running in pods.

Understanding Pod and Service Networking

Pod networking involves creating bridge networks on each node where pods receive individual namespaces, attach network interfaces, and obtain IP addresses from the node-specific subnet. This setup ensures that pods across different nodes can communicate via dedicated routes or overlay networks. However, rather than configuring pod-to-pod communication directly, you typically set up services. Services allow you to expose and reach applications with an assigned IP address and DNS name, simplifying internal and external access.

Recap: Types of Services

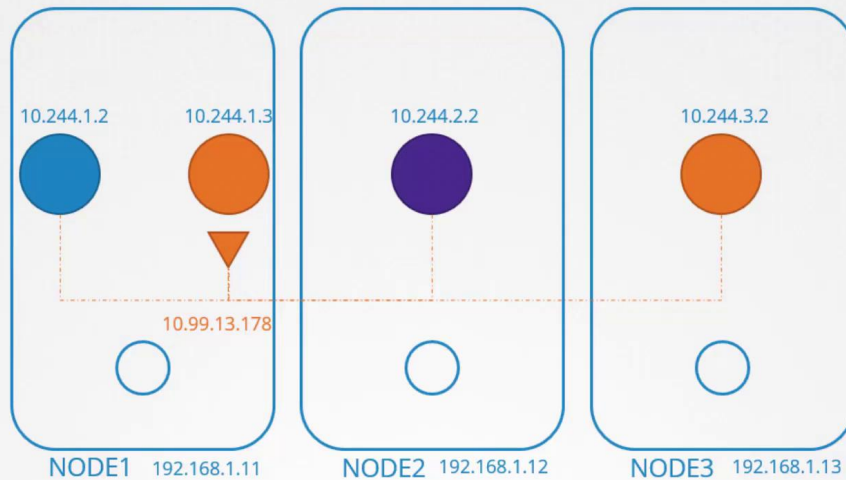
Imagine you have an "orange" pod that must be accessible to a "blue" pod. Instead of connecting the pods directly, you expose the orange pod with a service. This service is provided an IP address and DNS name, which the blue pod can use to establish a connection. When both pods reside on the same node, communication is straightforward; however, the process becomes more intricate when pods are distributed across multiple nodes.

Service Types: ClusterIP vs. NodePort

ClusterIP Services

By default, when a service is created, it is accessible to all pods in the cluster, regardless of the node they run on. This default service type, called ClusterIP, is ideal for applications meant strictly for internal access. For example, if your orange pod hosts a database, a ClusterIP service ensures internal connectivity.

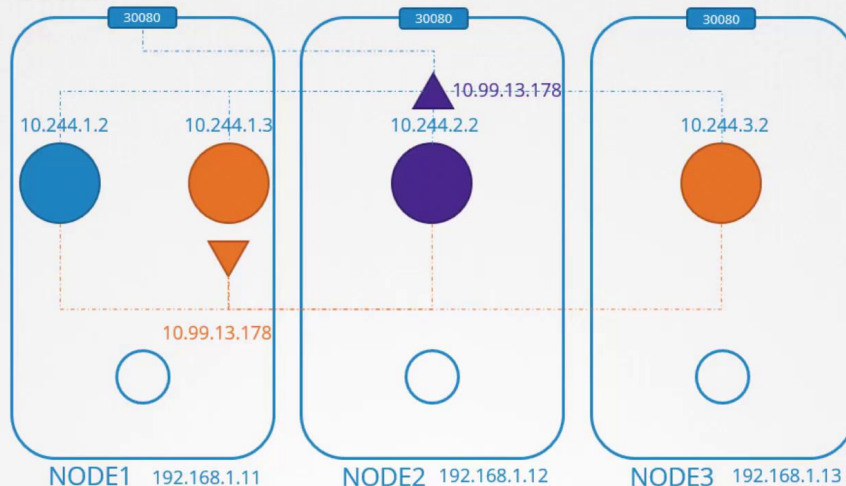
ClusterIP



NodePort Services

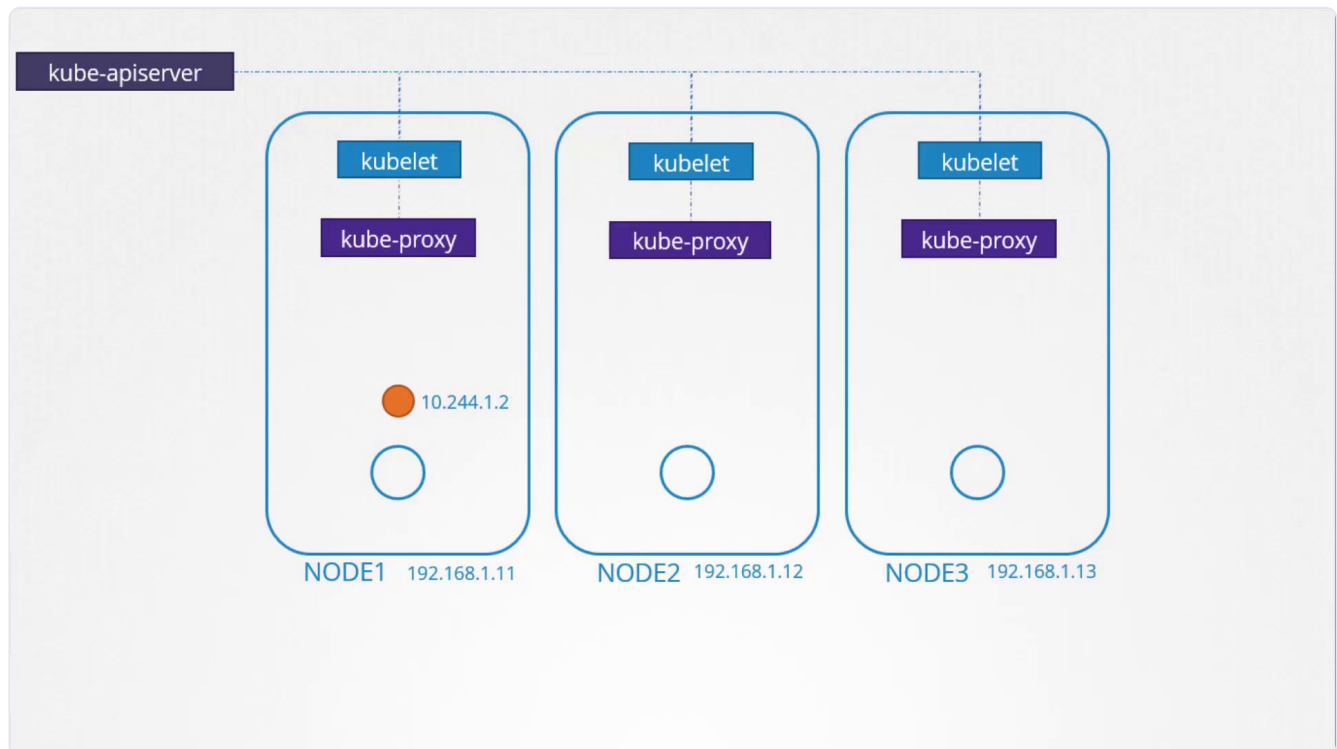
When you need a service to be accessible from outside the cluster, such as when a purple pod hosts a web application, you use a NodePort service. This service still receives an internal cluster IP for intra-cluster communication, but it also exposes the application on a specific port on every node. This allows external users or applications to reach the service without direct access to individual pod IPs.

NodePort

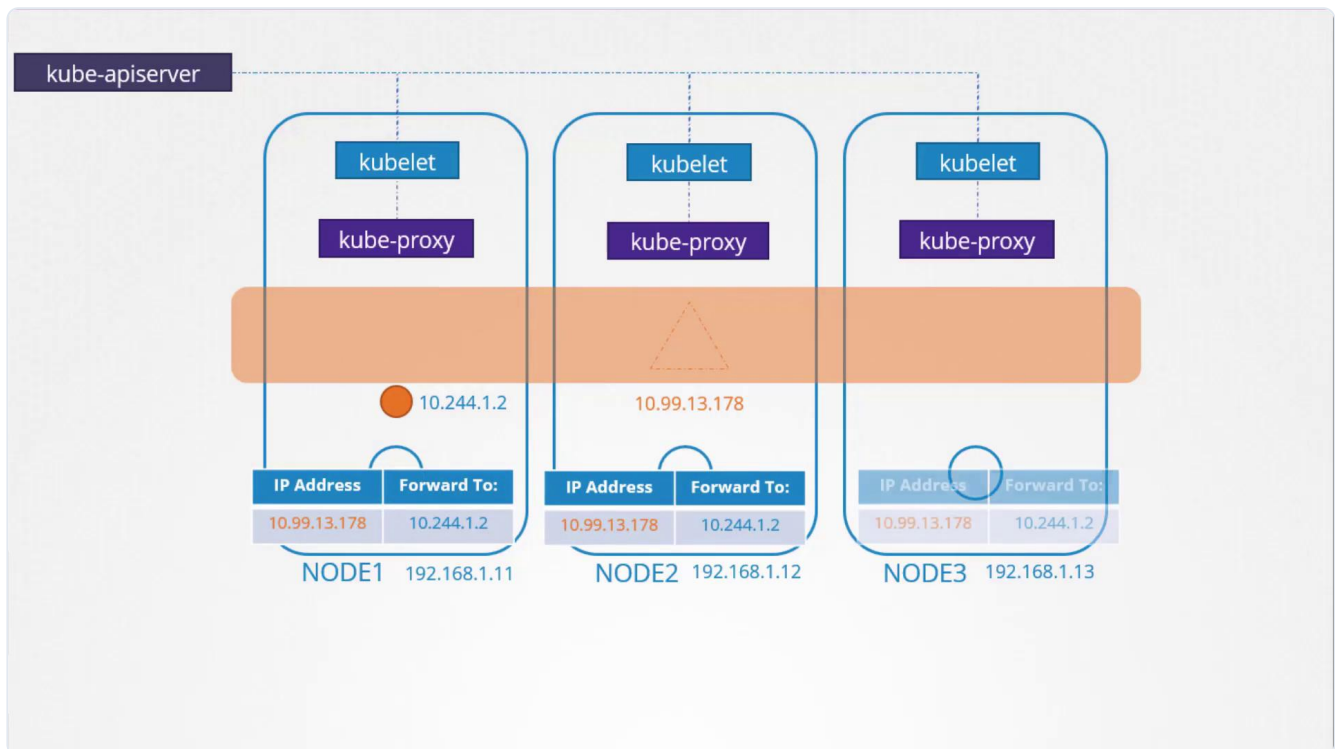


How Kubernetes Manages Service Networking

Let's start with a clean slate. Picture a three-node Kubernetes cluster that has no pods or services deployed yet. Each node runs a Kubelet process, responsible for pod creation by communicating with the Kube API Server and invoking the CNI plugin to configure networking. In parallel, every node also runs a Kube Proxy, which monitors cluster changes via the API Server and sets up forwarding rules whenever a new service is created.



Unlike pods that have their dedicated network interfaces, services are virtual constructs spanning the entire cluster. When you create a service object, Kubernetes automatically assigns it an IP address from a predefined range (set via the Kube API Server's `--service-cluster-ip-range` option). The kube-proxy on each node then configures the appropriate forwarding rules so that any traffic targeted at the service's IP and port is correctly routed to the backend pod.



Whenever a pod sends traffic to a service IP, the kube-proxy rules step in to forward that traffic to the chosen backend pod. These rules are dynamically updated as services are created or removed. Kube Proxy supports several proxy modes, including user space, iptables, and IPVS (with iptables being the default unless specified otherwise).

Here's an example command to start the kube-proxy with a specific proxy mode:

```
```bash theme={null} kube-proxy --proxy-mode [userspace | iptables | ipvs] ...
```

<Callout icon="lightbulb" color="#1CB2FE">

By default, kube-proxy uses iptables mode. Ensure your configuration specifies the intended mode using ...

## Practical Example: Service IP Assignment and Traffic Forwarding

Consider a scenario where a pod named "db" is running on node-1 with the IP address 10.244.1.2. To expose

```
```bash theme={null}
```

```
kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
db	1/1	Running	0	14h	10.244.1.2	node-1

```
kubectl get service
```

NAME	TYPE	CLUSTER-IP	PORT(S)	AGE
db-service	ClusterIP	10.103.132.104	3306/TCP	12h

In some clusters, the default service cluster IP range might be 10.0.0.0/24, but in other setups the API Server may be configured with a different range such as --service-cluster-ip-range=10.96.0.0/12. This means that service IPs can fall anywhere between 10.96.0.0 and 10.111.255.255. Meanwhile, the pod network CIDR might

be something like 10.244.0.0/16, ensuring that pod IP addresses range between 10.244.0.0 and 10.244.255.255. It is crucial that these ranges do not overlap.

When the "db-service" is created, kube-proxy establishes corresponding iptables rules so that traffic arriving at the service IP on port 3306 is redirected to the pod's IP on the same port:

```
```bash theme={null} iptables -L -t nat | grep db-service KUBE-SVC-XA5OGUC7YRHOS3PU tcp -- anywhere 10.103.132.104 / default/db-service: cluster IP / tcp dpt:3306 DNAT tcp -- anywhere anywhere / default/db-service: / tcp to:10.244.1.2:3306 KUBE-SEP-JBWCWHHQM57V2WN7 all -- anywhere anywhere / default/db-service: /
```

This DNAT (Destination Network Address Translation) rule ensures that any traffic to the service IP (10.103.132.104) is forwarded to the pod IP (10.244.1.2) on port 3306.

Similarly, a NodePort service would have iptables rules that forward traffic arriving on a designated port (e.g., 30080) to the pod IP on port 3306.

### ## Verifying kube-proxy Operation

To check which proxy mode your kube-proxy is using, you can inspect the kube-proxy logs:

```
```bash theme={null}
cat /var/log/kube-proxy.log
```

Note that the location of the kube-proxy log file may vary depending on your installation. If you do not see the expected entries, verify that the verbosity level of kube-proxy is set appropriately.

Conclusion

This lesson has provided a detailed overview of service networking in Kubernetes, explaining how services are assigned IP addresses, how kube-proxy forwards traffic, and the differences between ClusterIP and NodePort service types. Further details and related topics will be explored in subsequent content.

Happy learning!

For more information, check out the following resources:

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)
- [Terraform Registry](#)

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

DNS in kubernetes

This article explains how DNS is managed in Kubernetes, covering service and pod DNS records and their role in pod communication.

Welcome to this comprehensive guide on how DNS is managed within a Kubernetes cluster. In this article, we explore the mechanisms behind both service and pod DNS records, along with practical examples for enabling communication between pods. Before diving in, ensure you are familiar with the basics of DNS. If you're new to DNS concepts, please review the prerequisites below.

I Pre-Requisite

- ✓ What is DNS?
- ✓ Host/NS Lookup, Dig utility
- ✓ Recorded Types – A, CNAME
- ✓ Domain Name Hierarchy

Previously, we covered the fundamentals of DNS, including common tools such as `host`, `nslookup`, and `dig` alongside various DNS record types (A, CNAME, etc.) and the domain name hierarchy. We even demonstrated how to set up your own DNS server using CoreDNS. Now, we shift our focus to the DNS names assigned to various Kubernetes objects—like services and pods—and the different methods of accessing one pod from another.

Objectives

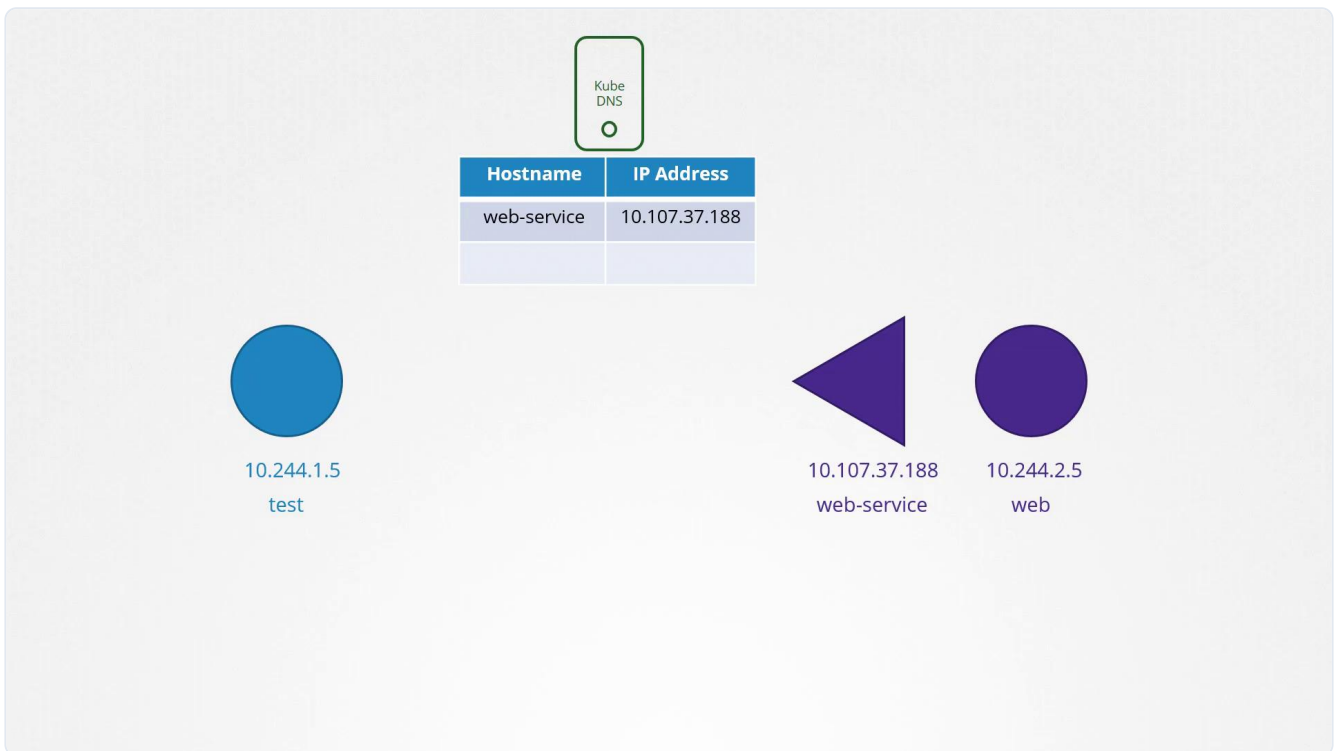
- ☐ What names are assigned to what objects?
- ☐ Service DNS records
- ☐ POD DNS Records

Imagine a three-node Kubernetes cluster with multiple pods and services distributed across them. Each node typically has a unique name and IP address registered in your organization's DNS server. However, our focus here is on the internal DNS resolution among the cluster's pods and services. By default, when you create a cluster, Kubernetes deploys a built-in DNS server (unless manually configured otherwise), which facilitates name resolution for pods and services.

Consider a simple scenario with two pods and a service in your cluster:

- A **test pod** with IP `10.244.1.5`.
- A **web pod** with IP `10.244.2.5`.

Even if these pods reside on different nodes (as indicated by their IP addresses), Kubernetes DNS assumes that all pods and services can be reached via their IP addresses. To allow the test pod to communicate with the web pod, a service named **web-service** is created. This service is assigned its own IP address (e.g., `10.107.37.188`) and automatically gets a DNS record mapping the service name to its IP.



Within the cluster, any pod can resolve and access the web service using its service name. For example, to access the web-service from the test pod, you could use:

```
```bash theme={null} curl http://web-service
```

## Output: Welcome to NGINX!

Earlier, we discussed namespaces in Kubernetes. Remember that pods within the same namespace (default namespace)

```
<Frame>
 ![The image illustrates two namespaces, each containing different individuals named Mark, highlighting the
</Frame>
```

In our scenario, because the test pod, web pod, and web-service are all in the **default** namespace, the

To illustrate DNS resolution with namespaces, consider the following examples:

```
```bash theme={null}
# When the service is in the default namespace
curl http://web-service
# When the service resides in the 'apps' namespace
curl http://web-service.apps
# Using the fully qualified domain name (FQDN)
curl http://web-service.apps.svc.cluster.local
# Output: Welcome to NGINX!
```

Each namespace in Kubernetes gets its own subdomain. All services within that namespace are grouped under a subdomain called "svc." Additionally, the entire cluster is associated with a root domain (by default, `cluster.local`). Thus, the fully qualified domain name for a service in the "apps" namespace is:

```
web-service.apps.svc.cluster.local
```

Now, let's discuss pod DNS records. By default, DNS records for pods are not created. However, this behavior can be explicitly enabled. When pod DNS records are activated, Kubernetes generates a DNS record for each pod by converting the pod's IP address into a hostname—replacing dots (`.`) with dashes (`-`). The record includes the pod's namespace, is set to type "pod," and utilizes the cluster's root domain.

For example, if a test pod in the default namespace has the IP `10.244.2.5`, the corresponding DNS record becomes:

```
10-244-2-5.apps.pod.cluster.local
```

This DNS entry resolves to the pod's IP address. You can test the resolution with the command below:

```
```bash theme={null} curl http://10-244-2-5.apps.pod.cluster.local
```

## Output: Welcome to NGINX!

---

```
```
```

For more detailed information on Kubernetes DNS and other concepts, consider reviewing the following resources:

- [Kubernetes Basics](#)
- [Kubernetes Documentation](#)
- [CoreDNS Documentation](#)

By understanding these DNS concepts, you can better manage communication within your Kubernetes cluster and ensure reliable service discovery in your environment.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

CoreDNS in Kubernetes

This lesson covers how Kubernetes uses CoreDNS for DNS resolution to facilitate communication between pods and services within a cluster.

Welcome to this lesson on CoreDNS in Kubernetes. In this guide, you will learn how Kubernetes implements DNS resolution within a cluster to facilitate seamless communication between pods and services.

In our previous lesson, we explored how to address a service or pod from another pod. Now, we will explain how Kubernetes leverages a centralized DNS server to achieve that functionality.

Imagine you have two pods with different IP addresses. One approach to enable communication between them is to add an entry into each pod's hosts file. For instance, on the first pod, you might map the second pod (named "web") to IP 10.244.2.5, and on the second pod, map the first pod (named "test") to IP 10.244.1.5. However, when dealing with thousands of pods that are frequently created and removed, manually managing these entries becomes impractical.

Instead of manually editing hosts files, Kubernetes deploys a central DNS server. Each pod is pre-configured via its `/etc/resolv.conf` file to use this centralized server (typically at 10.96.0.10), which automatically updates DNS records for new pods and services.

Kubernetes does not create DNS entries for individual pods manually. Instead, it sets up DNS records for services, and for pods, it converts IP addresses into hostnames by replacing dots with dashes.

Before Kubernetes version 1.12, this service was known as Kube-DNS. Starting with version 1.12, however, the recommended DNS server is CoreDNS, which brings enhanced flexibility and performance. Below is a conceptual illustration showing how pods configure their `/etc/resolv.conf` to point to the DNS server:

```
```bash theme={null} cat /etc/resolv.conf nameserver 10.96.0.10
```

### ## CoreDNS Setup in the Cluster

CoreDNS is deployed as a pod within the kube-system namespace. To ensure high availability, Kubernetes runs

CoreDNS requires a configuration file—commonly named "Corefile" and located at /etc/coredns/Corefile—that

```
```bash theme={null}
cat /etc/coredns/Corefile
.:53 {
    errors
    health
    kubernetes cluster.local in-addr.arpa ip6.arpa {
        pods insecure
        upstream
        fallthrough in-addr.arpa ip6.arpa
    }
    prometheus :9153
    proxy . /etc/resolv.conf
    cache 30
    reload
}
```

This configuration performs the following functions:

- Logs and handles errors.
- Provides health check endpoints.
- Integrates with Kubernetes via the Kubernetes plugin, configuring the primary domain to cluster.local and transforming pod IP addresses into a dashed hostname format.
- Exposes Prometheus metrics for monitoring.
- Forwards unresolved DNS queries (such as www.google.com) to the nameserver specified in the pod's /etc/resolv.conf.
- Caches DNS responses and supports dynamic reloads of the configuration upon changes.

Note that this configuration is stored in a ConfigMap. If adjustments are needed, simply update the ConfigMap:

```
```bash theme={null} kubectl get configmap -n kube-system NAME DATA AGE coredns 1 168d
```

Once the CoreDNS pod is running with the correct configuration, it continuously monitors the Kubernetes API

### ## DNS Service and Pod Configuration

To enable pods to communicate with the CoreDNS server, Kubernetes creates a service (named kube-dns by default)

```
```bash theme={null}
kubectl get service -n kube-system
NAME          TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
kube-dns      ClusterIP   10.96.0.10     <none>         53/UDP,53/TCP    1d
```

A typical pod's /etc/resolv.conf file will appear as:

```
```bash theme={null} cat /etc/resolv.conf
nameserver 10.96.0.10
search default.svc.cluster.local svc.cluster.local cluster.local
```

```

This configuration is automatically managed by the Kubelet. If you inspect the Kubelet configuration file,

```bash theme={null}
cat /var/lib/kubelet/config.yaml
...
clusterDNS:
- 10.96.0.10
clusterDomain: cluster.local

```

Resolving Services and Pods

With the correct DNS configuration, pods can resolve services using different domain name formats. For example, if you have a web service deployed in your cluster, you can access it using any of the following names:

- web-service
- web-service.default
- web-service.default.svc
- web-service.default.svc.cluster.local

These examples are demonstrated in the commands below:

```

```bash theme={null} cat /etc/resolv.conf nameserver 10.96.0.10 curl http://web-service curl http://web-
service.default curl http://web-service.default.svc curl http://web-service.default.svc.cluster.local

```

You can also use tools like nslookup or host to check the fully qualified domain name:

```

```bash theme={null}
host web-service
web-service.default.svc.cluster.local has address 10.107.37.188

```

The search entries in /etc/resolv.conf allow the resolver to append the listed domains when you use partial names. However, individual pod names must always be resolved with their complete fully qualified domain name (FQDN).

For example:

```

```bash theme={null} cat /etc/resolv.conf nameserver 10.96.0.10 search default.svc.cluster.local svc.cluster.local
cluster.local

```

```

host web-service web-service.default.svc.cluster.local has address 10.107.37.188

```

```

host 10-244-2-5 Host 10-244-2-5 not found: 3(NXDOMAIN)

```

```

host 10-244-2-5.default.pod.cluster.local 10-244-2-5.default.pod.cluster.local has address 10.244.2.5 ```

```

The search entries in the /etc/resolv.conf file simplify service resolution by allowing shorter names. However, pod-specific DNS records require their full FQDN for proper resolution.

## Conclusion

---

By deploying CoreDNS and configuring all pods to use the centralized DNS service, Kubernetes enables dynamic and automated DNS resolution for both services and pods. This robust setup eliminates the need for manual host file configuration, thereby streamlining communication across the cluster.

Revisit and practice these concepts to deepen your understanding of managing DNS within a Kubernetes environment. Happy learning!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

# Ingress

This article provides a comprehensive guide on Kubernetes Ingress, covering its configuration, deployment, and use cases for managing external access to applications.

Welcome to this comprehensive guide on Kubernetes Ingress. In this article, we will explore the differences between Services and Ingress, explain when to use each, and demonstrate how to deploy and configure Ingress resources effectively. We begin by revisiting Services and then build toward understanding Ingress.

## Background: Using Services for External Access

Imagine you are deploying an application for a company with an online store at [myonlinestore.com](https://myonlinestore.com). You build your application as a Docker image and deploy it on a Kubernetes cluster as a pod within a Deployment. The application requires a database, so you deploy a MySQL pod and expose it with a ClusterIP Service named MySQL service. Internally, your application functions properly.

To expose the application externally, you create a Service of type NodePort, mapping the application to a high port (e.g., 38080) on your cluster nodes. In this configuration, users access your application using a URL like:

`http://<node-ip>:38080`

As traffic increases, the Service load-balances the requests among multiple application pods.

In a production environment, however, you likely want users to access your application through a user-friendly domain name instead of a node IP address with a high port number. To achieve this, you would update your DNS configuration to point to your node IPs and deploy a proxy server that forwards requests from standard port 80 (or 443 for HTTPS) on your DNS to the NodePort defined in your cluster. With this approach, users can simply navigate to [myonlinestore.com](https://myonlinestore.com).

### Cloud-Native Approach with LoadBalancer

If your application is hosted on a public cloud platform like [Google Cloud Platform \(GCP\)](https://cloud.google.com/), the process can be simplified further. Instead of creating a NodePort Service, you can deploy a Service of type LoadBalancer. In this setup:

- Kubernetes assigns an internal high port.
- It sends a request to GCP to deploy a network load balancer.
- The cloud load balancer routes traffic to the internal port across all nodes.

- An external IP is provided by the load balancer, which you point your DNS to.

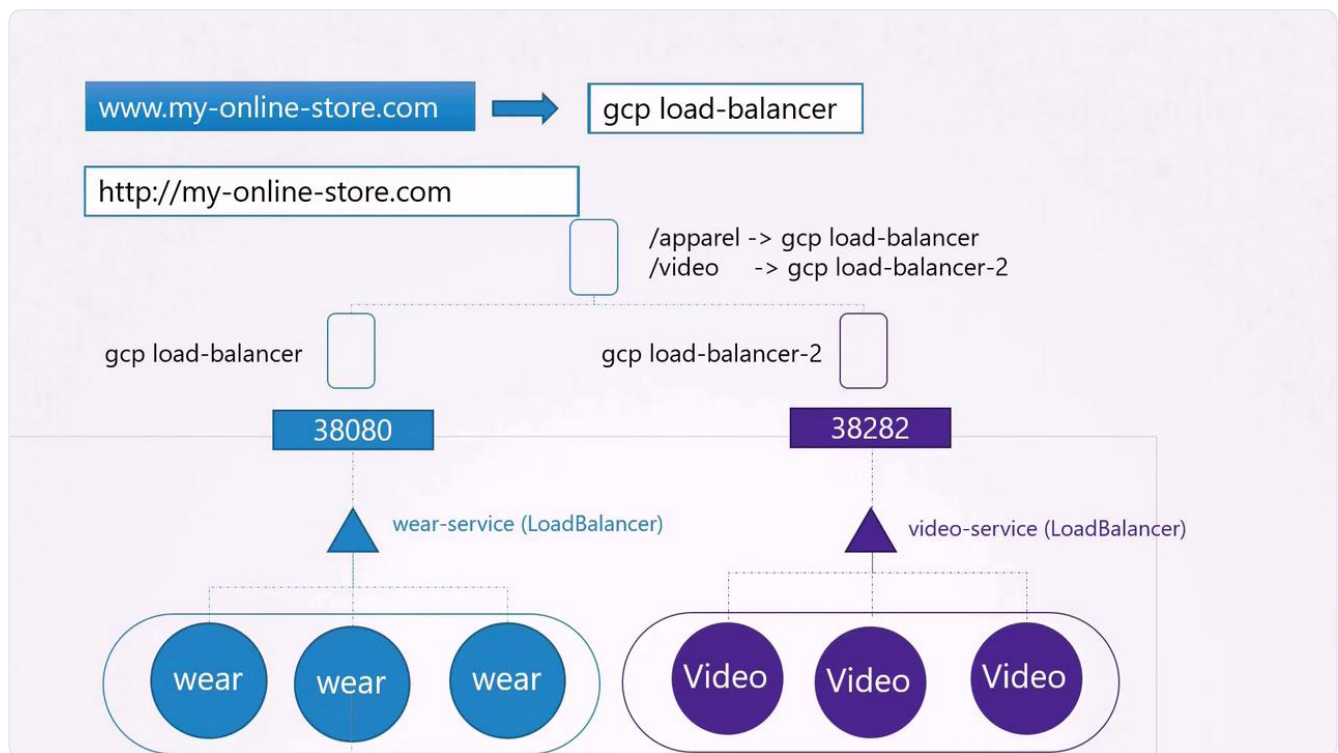
This allows users to access your application directly using myonlinestore.com.

## The Need for Ingress

Imagine your company expands and you launch new services. For example, you might offer:

- A video streaming service on myonlinestore.com/watch
- The original application on myonlinestore.com/wear

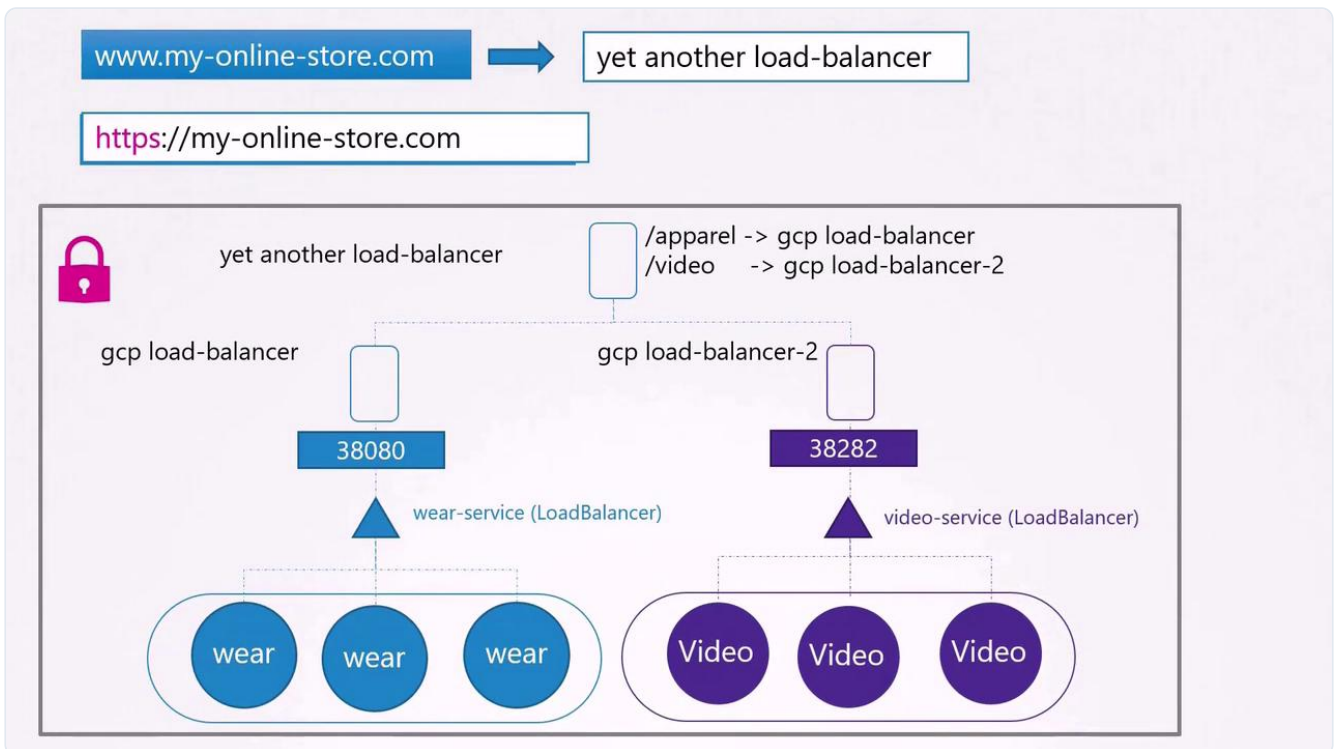
Even if both applications run within the same cluster with separate Deployments and Services (e.g., a LoadBalancer Service for the video service), each service might get its own high port and cloud load balancer. Managing multiple load balancers can increase costs, add complexity, and complicate SSL/TLS (HTTPS) configurations.



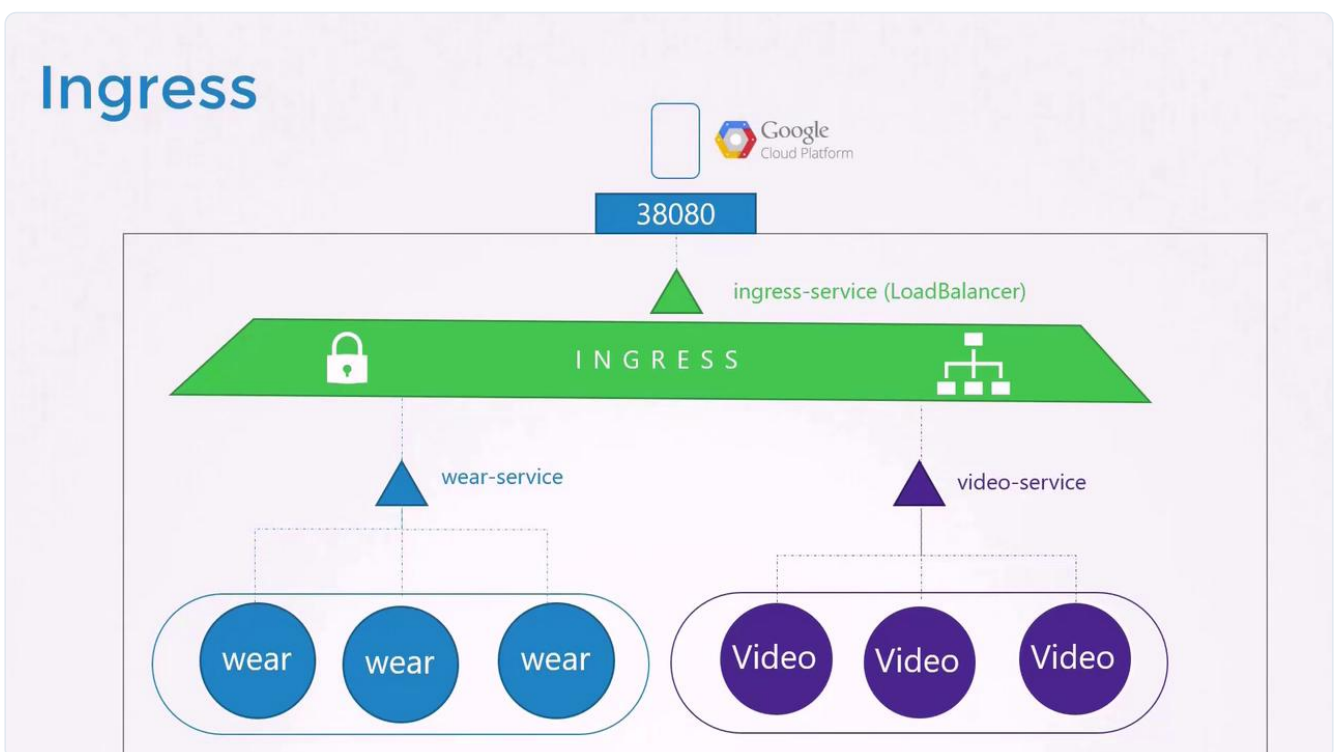
## Introducing Ingress

Ingress simplifies external access by providing a single externally accessible IP for your Kubernetes applications. It allows you to configure URL-based routing rules, SSL termination, authentication, and more—acting as a built-in layer 7 load balancer.

Even with Ingress, you still need an initial exposure mechanism (via NodePort or a cloud-native LoadBalancer). However, once this is set up, all further changes are made through the Ingress controller.

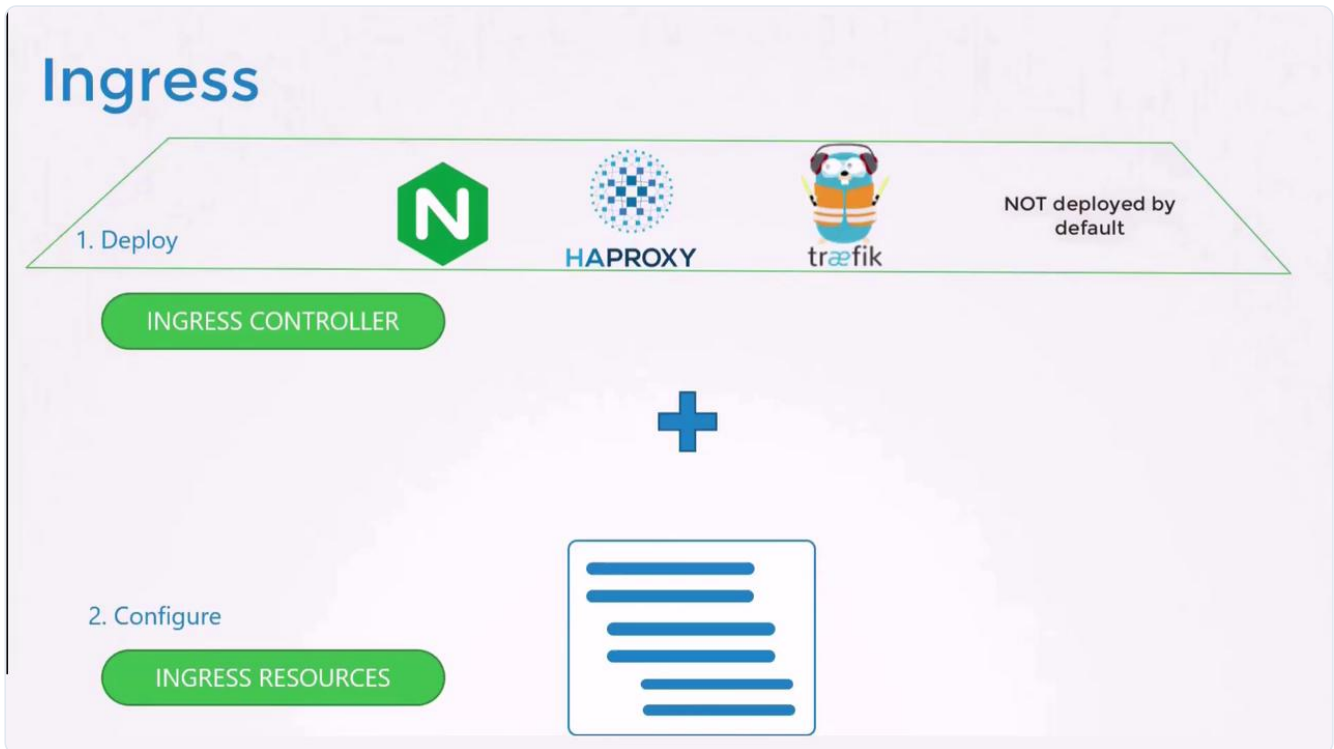


Without Ingress, you would have to manually deploy and configure a reverse proxy or load balancer (such as NGINX, HAProxy, or Traefik) within your cluster to handle URL routing and manage SSL certificates. Ingress builds on these principles to offer an integrated solution.



## Deploying an Ingress Controller

To use Ingress, you must first deploy an Ingress controller. The controller continuously monitors the cluster for changes in Ingress resources and reconfigures the underlying load balancing solution accordingly.



A Kubernetes cluster does not include an Ingress controller by default. If you create Ingress resources without deploying an Ingress controller, they will have no effect.

There are several Ingress controllers available, including GCE, NGINX, Contour, HAProxy, Traefik, and Istio. In this guide, we will use NGINX as the example.

## Deploying the NGINX Ingress Controller

Below is an example of an NGINX Ingress controller deployment:

```
```yaml
theme={null}
apiVersion: extensions/v1beta1
kind: Deployment
metadata:
  name: nginx-ingress-controller
spec:
  replicas: 1
  selector:
    matchLabels:
      name: nginx-ingress
  template:
    metadata:
      labels:
        name: nginx-ingress
    spec:
      containers:
        - name: nginx-ingress-controller
          image: quay.io/kubernetes-ingress-controller/nginx-ingress-controller:0.21.0
          args:
            - /nginx-ingress-controller
```
```

This Deployment creates one replica of the NGINX Ingress controller using a Kubernetes-optimized NGINX image. To decouple configuration data from the image, you create a ConfigMap. This allows easy updates to log paths. Below is an example Service to expose the NGINX Ingress controller using NodePort:

```
```yaml theme={null}
apiVersion: v1
kind: Service
metadata:
  name: nginx-ingress
spec:
  type: NodePort
  ports:
    - port: 80
      targetPort: 80
      protocol: TCP
      name: http
    - port: 443
      targetPort: 443
      protocol: TCP
      name: https
  selector:
    name: nginx-ingress
```

For a more comprehensive configuration, you can include environment variables to pass the Pod's name and namespace to the container. This enables the Ingress controller to load its configuration dynamically. Here is a complete configuration that includes the Deployment, Service, ConfigMap, and ServiceAccount:

```
```yaml theme={null} apiVersion: extensions/v1beta1 kind: Deployment metadata: name: nginx-ingress-controller
spec: replicas: 1 selector: matchLabels: name: nginx-ingress template: metadata: labels: name: nginx-ingress
spec: containers: - name: nginx-ingress-controller image: quay.io/kubernetes-ingress-controller/nginx-ingress-controller:0.21.0 args: - /nginx-ingress-controller - --configmap=$(POD_NAMESPACE)/nginx-configuration env: -
name: POD_NAME valueFrom: fieldRef: fieldPath: metadata.name - name: POD_NAMESPACE valueFrom: fieldRef: fieldPath: metadata.namespace ports: - name: http containerPort: 80 - name: https containerPort: 443
```

---

```
apiVersion: v1 kind: Service metadata: name: nginx-ingress spec: type: NodePort ports: - port: 80 targetPort: 80
protocol: TCP name: http - port: 443 targetPort: 443 protocol: TCP name: https selector: name: nginx-ingress
```

---

```
kind: ConfigMap apiVersion: v1 metadata: name: nginx-configuration
```

---

```
apiVersion: v1 kind: ServiceAccount metadata: name: nginx-ingress-serviceaccount
```

---

In addition to the objects above, you must create the necessary Roles, ClusterRoles, and RoleBindings so that the NGINX Ingress controller can manage the Ingress resources.

## ## Creating Ingress Resources

Once the NGINX Ingress controller is deployed, you can begin creating Ingress resources that define rules for routing traffic to Services.

### ### Simple Ingress for a Single Service

The following Ingress resource routes all incoming traffic to a single backend Service named "wear-service".

```
```yaml
theme={null}
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
  name: ingress-wear
spec:
  backend:
    serviceName: wear-service
    servicePort: 80
```
```

To create this Ingress resource, run:

```
```bash
theme={null} kubectl create -f Ingress-wear.yaml
```
```

You should see output similar to:

```
```bash
theme={null}
kubectl create -f Ingress-wear.yaml
ingress.extensions/ingress-wear created
```
```

Verify its creation with:

```
```bash
theme={null} kubectl get ingress
```
```

Expected output:

```
```bash
theme={null}
kubectl get ingress
NAME          HOSTS    ADDRESS    PORTS    AGE
ingress-wear  *        <none>     80       2s
```
```

This configuration directs all traffic to "wear-service".

## Ingress with Multiple URL Paths

For more complex routing—such as directing traffic from different URL paths to different backend Services—use Ingress rules. Suppose you want:

- Traffic to myonlinestore.com/wear to go to "wear-service"
- Traffic to myonlinestore.com/watch to go to "watch-service"

Define an Ingress resource as follows:

```
```yaml theme={null} apiVersion: extensions/v1beta1 kind: Ingress metadata: name: ingress-wear-watch spec:
rules: - http: paths: - path: /wear backend: serviceName: wear-service servicePort: 80 - path: /watch backend:
serviceName: watch-service servicePort: 80
```

After creating this resource, you can view its details by running:

```
```bash theme={null}
kubectl describe ingress ingress-wear-watch
```

The output will display the rules and backend configurations similar to:

```
```bash theme={null} Name: ingress-wear-watch Namespace: default Address:
Default backend: default-http-backend:80 () Rules: Host Path Backends ---- ---- ----- * /wear wear-service:80
() /watch watch-service:80 () Annotations: Events: Type Reason Age From Message ---- ----- ---- -----
Normal CREATE 14s nginx-ingress-controller Ingress default/ingress-wear-watch
```

If a user visits an undefined URL (e.g., myonlinestore.com/listen), you can configure a default backend to handle the request.

Ingress Based on Host Names

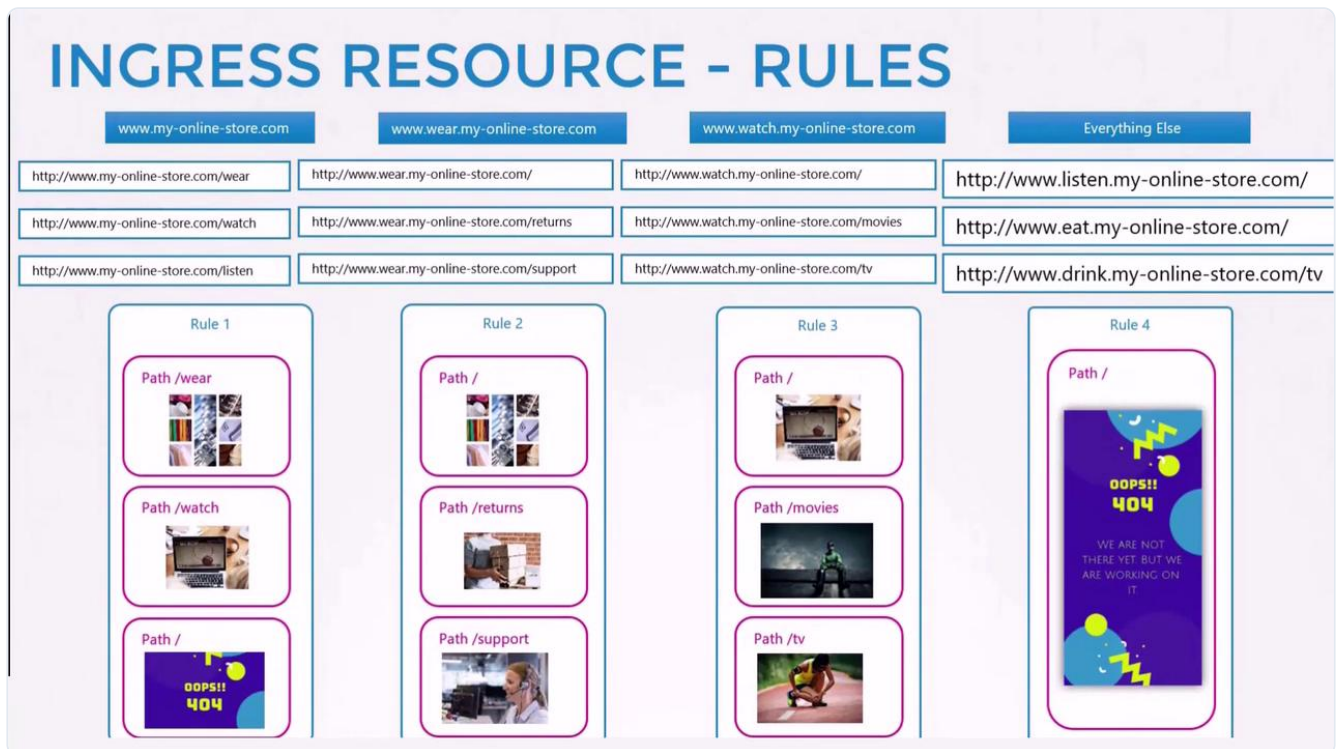
Another common scenario involves routing traffic based on host names. For example, you might want:

- * Traffic for myonlinestore.com to go to "primary-service"
- * Traffic for www.myonlinestore.com to go to "secondary-service"

Here's how you can define the Ingress resource with host-specific rules:

```
```yaml theme={null}
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
 name: ingress-domain-routing
spec:
 rules:
 - host: myonlinestore.com
 http:
 paths:
 - path: /
 backend:
 serviceName: primary-service
 servicePort: 80
 - host: www.myonlinestore.com
 http:
 paths:
 - path: /
 backend:
 serviceName: secondary-service
 servicePort: 80
```

In this example, each rule handles traffic for a specific domain by routing all requests ("/") to the designated backend Service.



Splitting traffic by URL involves one rule with multiple paths, whereas splitting traffic by domain requires multiple rules with specific host fields. If you do not define a host, the Ingress rule will match all incoming traffic, regardless of the domain.

## Conclusion

This guide has walked through the process of setting up and configuring Ingress in Kubernetes. We explored a variety of use cases—from simple backend routing to complex rules based on URL paths and host names. The Ingress controller simplifies external access to your cluster, helps manage SSL/TLS termination, and consolidates your load balancing configuration.

In practice tests or production environments, you may encounter labs where the Ingress controller and applications are pre-deployed, or more challenging labs where you need to deploy the Ingress controller and resources from scratch.

Good luck with your Kubernetes projects, and we hope this article has been helpful in advancing your understanding of Kubernetes Ingress.

For additional information, consider reviewing:

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)
- [NGINX Ingress Controller Repository](#)

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Introduction to Gateway API 2025 Updates

This article introduces the Gateway API and its updates, addressing Ingress limitations and enhancing traffic management capabilities in Kubernetes.

In this article, we introduce the Gateway API and explain how it addresses the challenges posed by the traditional Ingress resource. Previously, when using Ingress, multiple teams or organizations sharing a single Ingress resource faced coordination challenges. For instance, if team A manages a web service and team B manages a video service, they would need to coordinate their changes on one Ingress resource. This multi-tenancy issue is a significant limitation of Ingress, which can only be managed by one team at a time.

## Limitations of Ingress

Consider the basic Ingress configuration below, which routes traffic based on host names:

```
```yaml theme={null}
```

ingress-wear-watch.yaml

```
apiVersion: extensions/v1beta1 kind: Ingress metadata: name: ingress-wear-watch spec: rules: - host: wear.my-online-store.com http: paths: - backend: serviceName: wear-service servicePort: 80 - host: watch.my-online-store.com http: paths: - backend: serviceName: watch-service servicePort: 80
```

This configuration supports HTTP-based rules such as host and path matching. However, it does not natively

```
```yaml theme={null}
ingress-with-annotations.yaml
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
 name: ingress-wear-watch
 annotations:
 nginx.ingress.kubernetes.io/ssl-redirect: "true"
 nginx.ingress.kubernetes.io/force-ssl-redirect: "true"
spec:
 rules:
 - host: wear.my-online-store.com
 http:
 paths:
 - path: /foo
 backend:
 serviceName: wear-service
 servicePort: 80
 - host: watch.my-online-store.com
 http:
 paths:
 - backend:
 serviceName: watch-service
 servicePort: 80
```

Because each controller implements its own set of annotations, these configurations are tightly coupled to specific controllers and cannot be validated by Kubernetes itself.

Similarly, advanced scenarios like configuring Cross-Origin Resource Sharing (CORS) require different annotations depending on the controller. For example, a CORS configuration for NGINX might look like this:

```
```yaml theme={null}
```

Ingress-cors.yaml (for NGINX)

```
apiVersion: networking.k8s.io/v1 kind: Ingress metadata: name: cors-ingress annotations:
nginx.ingress.kubernetes.io/enable-cors: "true" nginx.ingress.kubernetes.io/cors-allow-methods: "GET, PUT,
POST" nginx.ingress.kubernetes.io/cors-allow-origin: "https://allowed-origin.com" nginx.ingress.kubernetes.io/
cors-allow-credentials: "true"
```

And for a traffic controller like Traefik, a similar configuration would be:

```
```yaml theme={null}
Ingress-traefik.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
 name: traefik-ingress
 annotations:
 # CORS Configuration
 traefik.ingress.kubernetes.io/headers.customresponseheaders: |
 Access-Control-Allow-Origin: '*'
 Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
 Access-Control-Allow-Headers: Content-Type, Authorization
 Access-Control-Allow-Credentials: true
 Access-Control-Max-Age: 3600
```

These examples clearly demonstrate that the same use case leads to different configurations based solely on the chosen controller.

## Introducing the Gateway API

The Gateway API was created as an official Kubernetes project to overcome the limitations of Ingress. It supports both layer 4 (transport) and layer 7 (application) routing, representing the next generation of load balancing and service mesh APIs. By decoupling responsibilities, the Gateway API introduces three distinct objects:

1. **Gateway Class:** Configured by infrastructure providers to define the underlying network infrastructure (e.g., NGINX, Traefik, or other load balancers).
2. **Gateway:** Managed by cluster operators; these are instances built from a Gateway Class.
3. **HTTPRoute (and other route types):** Managed by application developers; these routes support various protocols such as HTTP, TCP, and gRPC.

Unlike Ingress, the Gateway API offers a unified and declarative configuration that is independent of controller-specific annotations.

### Gateway API Configuration Example

The following example illustrates how to define a Gateway Class, a Gateway, and an HTTPRoute using the Gateway API:

```
```yaml theme={null}
```

gateway-class.yaml

```
apiVersion: gateway.networking.k8s.io/v1 kind: GatewayClass metadata: name: example-class spec:
controllerName: example.com/gateway-controller
```

```
```yaml theme={null}
gateway.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
 name: example-gateway
spec:
 gatewayClassName: example-class
 listeners:
 - name: http
 protocol: HTTP
 port: 80
```

```
```yaml theme={null}
```

http-route.yaml

```
apiVersion: gateway.networking.k8s.io/v1 kind: HTTPRoute metadata: name: example-httproute spec:
parentRefs: - name: example-gateway hostnames: - "www.example.com" rules: - matches: - path: type:
PathPrefix value: /login backendRefs: - name: example-svc port: 8080
```

In this configuration, the HTTPRoute routes requests with a path prefix of "/login" arriving at "www.example.com".

TLS Configuration with Gateway API

Traditional Ingress configurations handle TLS by using the `spec.tls` section, often accompanied by additional annotations.

```
```yaml theme={null}
ingress-secure-app.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
 name: secure-app
 annotations:
 nginx.ingress.kubernetes.io/ssl-redirect: "true"
 nginx.ingress.kubernetes.io/force-ssl-redirect: "true"
spec:
 tls:
 - hosts:
 - secure.example.com
 secretName: tls-secret
```

In contrast, the Gateway API allows for a more structured TLS configuration as part of the listener definition:

```
```yaml theme={null}
```

gateway-secure.yaml

```
apiVersion: gateway.networking.k8s.io/v1 kind: Gateway metadata: name: secure-gateway spec:
gatewayClassName: example-gc listeners: - name: https port: 443 protocol: HTTPS tls: mode: Terminate
certificateRefs: - kind: Secret name: tls-secret allowedRoutes: kinds: - kind: HTTPRoute
```

Here, the HTTPS listener on port 443 is explicitly configured with TLS termination, referencing the necessary certificate and key secrets.

Traffic Splitting and Canary Deployments

Ingress relies on annotations for complex use cases like canary deployments. For example, you might use NGINX Ingress Controller annotations to configure traffic splitting.

```
```yaml theme={null}
canary-ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
 name: canary-ingress
 annotations:
 nginx.ingress.kubernetes.io/canary: "true"
 nginx.ingress.kubernetes.io/canary-weight: "20"
spec:
 rules:
 - http:
 paths:
 - path: /
 pathType: Prefix
 backend:
 service:
 name: app-v2
 port:
 number: 80
```

The Gateway API simplifies this process by natively defining traffic splitting within an HTTPRoute:

```
```yaml theme={null}
```

app-gateway.yaml

```
apiVersion: gateway.networking.k8s.io/v1 kind: HTTPRoute metadata: name: split-traffic spec: parentRefs: -
name: app-gateway rules: - backendRefs: - name: app-v1 port: 80 weight: 80 - name: app-v2 port: 80 weight: 20
```

In this example, "app-v1" receives 80% of the traffic while "app-v2" handles the remaining 20%. This declarative

Centralized Advanced Configuration

For features like Cross-Origin Resource Sharing (CORS), the Gateway API centralizes configurations without

```
```yaml theme={null}
http-route-cors.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
 name: cors-route
spec:
 parentRefs:
 - name: my-gateway
 rules:
 - matches:
 - path:
 type: PathPrefix
 value: /api
 filters:
 - type: ResponseHeaderModifier
 responseHeaderModifier:
 add:
 - name: Access-Control-Allow-Origin
 value: "*"
 - name: Access-Control-Allow-Methods
 value: "GET, POST, PUT, DELETE, OPTIONS"
 - name: Access-Control-Allow-Headers
 value: "Content-Type,Authorization"
 - name: Access-Control-Allow-Credentials
 value: "true"
 - name: Access-Control-Max-Age
 value: "3600"
 backendRefs:
 - name: api-service
```

This self-contained configuration is consistent and works seamlessly across different Gateway API controllers.

## Gateway API Controller Support

A range of major controllers now support, or are actively implementing, the Gateway API. This industry support includes platforms such as Amazon EKS, Azure Application Gateway for Containers, Contour, Envoy, Google Kubernetes Engine, HAProxy, Istio, Kong, Kuma, NGINX, and many others. Such broad adoption underlines the growing momentum and reliability of the Gateway API.

# Gateway API

Object	OSI Layer	Routing Discriminator	TLS Support	Purpose
HTTPRoute	Layer 7	Anything in the HTTP Protocol	Terminated only	HTTP and HTTPS Routing
TLSRoute	Somewhere between layer 4 and 7	SNI or other TLS properties	Passthrough or Terminated	Routing of TLS protocols including HTTPS where inspection of the HTTP stream is not required.
TCPRoute	Layer 4	destination port	Passthrough or Terminated	Allows for forwarding of a TCP stream from the Listener to the Backends
UDPRoute	Layer 4	destination port	None	Allows for forwarding of a UDP stream from the Listener to the Backends.
GRPCRoute	Layer 7	Anything in the gRPC Protocol	Terminated only	gRPC Routing over HTTP/2 and HTTP/2 cleartext

# Gateway Controller Implementation

## Gateway Controller Implementation Status

- [Acnodal EPIC](#)
- [Airlock Microgateway](#)
- [Amazon Elastic Kubernetes Service \(GA\)](#)
- [Apache APISIX \(beta\)](#)
- [Avi Kubernetes Operator \(tech preview\)](#)
- [Azure Application Gateway for Containers \(GA\)](#)
- [Cilium \(beta\)](#)
- [Contour \(GA\)](#)
- [Easegress \(GA\)](#)
- [Emissary-Ingress \(Ambassador API Gateway\) \(alpha\)](#)
- [Envoy Gateway \(GA\)](#)
- [Flomesh Service Mesh \(beta\)](#)
- [Gloo Gateway \(GA\)](#)
- [Google Kubernetes Engine \(GA\)](#)

- [HAProxy Ingress \(alpha\)](#)
- [HAProxy Kubernetes Ingress Controller \(GA\)](#)
- [HashiCorp Consul](#)
- [Istio \(GA\)](#)
- [Kong Ingress Controller \(GA\)](#)
- [Kong Gateway Operator \(GA\)](#)
- [Kuma \(GA\)](#)
- [LiteSpeed Ingress Controller](#)
- [LoxiLB \(beta\)](#)
- [NGINX Gateway Fabric \(GA\)](#)
- [ngrok \(preview\)](#)
- [STUNner \(beta\)](#)
- [Traefik Proxy \(GA\)](#)
- [Tyk \(work in progress\)](#)
- [WSO2 APK \(GA\)](#)

## Conclusion

The Gateway API overcomes the limitations of Ingress by providing a more declarative, structured, and multi-protocol solution for traffic management. By segregating responsibilities among GatewayClass, Gateway, and HTTPRoute, it simplifies multi-tenant management and advanced routing scenarios such as TLS termination, traffic splitting, and CORS configuration.

For a deeper understanding of these concepts, try practicing with hands-on labs to experiment with different Gateway API configurations.